



ΠΑΝΕΠΙΣΤΗΜΙΟ
ΠΑΤΡΩΝ
UNIVERSITY OF PATRAS



DEPARTMENT OF ELECTRICAL AND COMPUTER ENGINEERING

Submission to the
5th AI-Hub Artificial Intelligence Contest (2026)
for undergraduate students of the University of Patras

Sign Language Time Series Analysis Using
Machine Learning Methods

Spyridon Bantis

Undergraduate student, Department of Electrical and Computer Engineering
University of Patras

Patras, September 2026

Contents

1	Introduction	5
1.1	Sign Language Recognition	5
1.2	Motivation and Problem Statement	6
1.3	Proposed Approach	7
1.4	Report Structure	8
2	Theoretical Background	10
2.1	Sign Language and the Recognition Problem	10
2.1.1	Isolated vs. Continuous Sign Language Recognition . .	11
2.1.2	Challenges in Sign Language Recognition	11
2.2	Skeleton-based Representations and Pose Estimation	13
2.2.1	MediaPipe Holistic	13
2.2.2	Skeletal Preprocessing	14
2.2.3	Advantages and Limitations	15
2.3	Self-Supervised and Metric Learning	15
2.3.1	Representation Learning and Embedding Spaces	15
2.3.2	Pair-based Losses	16
2.3.3	Proxy-based Losses	16
2.3.4	Self-Supervised Embedding Learning	17
2.4	Proxy Anchor Loss	18
2.4.1	Core Idea	18
2.4.2	Loss Formulation	19
2.4.3	Hyperparameters	22
2.4.4	Advantages over Triplet Loss	23
2.4.5	Limitations of Proxy Anchor Loss	23
2.5	Transformer Architecture for Sequence Modeling	25
2.5.1	Self-Attention Mechanism	25
2.5.2	Transformer Encoder	26
2.5.3	Positional Encoding	26
2.5.4	Transformer for Sign Language Recognition	26
2.6	Hypergraph Transformer for Skeleton-based Action Recognition	29

2.6.1	From Graphs to Hypergraphs	29
2.6.2	Key Differences from Standard Graph Convolution . . .	30
2.6.3	Hyperformer Architecture	31
2.6.4	Joint Grouping for MediaPipe	31
2.6.5	Limitations for Sign Language Recognition	31
2.7	The WLASL Dataset	32
2.7.1	Dataset Description	32
2.7.2	Data Splits	33
2.7.3	Challenges	33
2.7.4	Evaluation Protocol	34
3	State of the Art	35
3.1	Isolated Sign Language Recognition	35
3.2	Appearance-based Methods	36
3.3	Skeleton-based Methods	36
3.4	Benchmarks for Isolated Sign Language Recognition	37
3.5	Positioning of This Work	38
4	Geometric Distance Measures for Skeletal Poses	39
4.1	Problem Formulation	39
4.2	Euclidean Distance (L2)	40
4.3	Procrustes Distance	40
4.3.1	Rotation-Regularized Procrustes	40
4.4	Articulated Pose Distance (APD)	42
4.4.1	Bone Orientation as Quaternions	42
4.4.2	Geodesic Distance and Weighting	42
4.4.3	Hierarchical Alignment Propagation	43
4.5	Complementarity of the Distance Measures	43
5	Methodology	44
5.1	Overview of the Proposed Framework	44
5.2	Data Preprocessing Pipeline	45
5.2.1	Skeleton Extraction	46
5.2.2	Temporal Normalization	46
5.2.3	Coordinate Standardization	46
5.2.4	Hand Joint Extraction for Encoder Pretraining	47
5.3	Distance Encoder Pretraining with Proxy Anchor Loss	47
5.3.1	Encoder Architecture	48
5.3.2	Proxy Anchor Pretraining Procedure	48
5.3.3	Frame-level vs. Sequence-level Training	49
5.3.4	Frozen vs. Fine-tuned Encoder	50

5.4	Encoder Fusion Strategies	50
5.4.1	Single Encoder Integration	50
5.4.2	Multi-Encoder Fusion	50
5.4.3	Evaluated Configurations	51
5.5	Transformer-based Recognition Model	52
5.5.1	Input Representation	52
5.5.2	Transformer Encoder	53
5.5.3	Classification Head	53
5.5.4	Training Procedure	54
5.6	Hyperformer Adaptation for WLASL	54
5.6.1	Input Format Adaptation	55
5.6.2	Skeleton Graph Definition	55
5.6.3	Joint Grouping	56
5.6.4	Hyperformer with Proxy Anchor Loss	56
5.6.5	Implementation Constraints	56
5.7	Implementation Details	57
5.7.1	Software and Frameworks	57
5.7.2	Hardware	57
5.7.3	Reproducibility	58
6	Experiments and Results	59
6.1	Experimental Setup	59
6.1.1	Datasets	59
6.1.2	Evaluation Metrics	60
6.1.3	Baselines and Comparisons	60
6.1.4	Evaluated Configurations	61
6.2	Results and Analysis	61
6.2.1	Comparison with the Geometric Baselines	62
6.2.2	Effect of the Vocabulary Size	64
6.2.3	Scalability to WLASL-2000	65
6.2.4	Effect of Fusion Strategy	67
6.2.5	Effect of Freezing the Encoder	67
6.2.6	Hyperformer Results	67
6.3	Comparison with State-of-the-Art Methods	69
6.3.1	Discussion of Similarities and Differences	69
7	Conclusions and Future Work	71
7.1	Summary of the Work	71
7.2	Main Findings	71
7.3	Limitations	72
7.4	Future Work	73

7.5 Concluding Remarks	74
A Qualitative Demonstration	75

Chapter 1

Introduction

1.1 Sign Language Recognition

Sign language is a complete and natural language used by Deaf and hard-of-hearing communities worldwide as their primary means of communication. Unlike spoken languages, sign languages are visual-gestural systems that convey meaning through the coordinated movement of hands, arms, facial expressions, and body posture. Each country has developed its own sign language with unique grammar, syntax, and vocabulary — for instance, American Sign Language (ASL), Greek Sign Language (GSL), and British Sign Language (BSL) are entirely distinct linguistic systems, not derived from their spoken counterparts.

The global Deaf community comprises an estimated 70 million people, according to the World Federation of the Deaf. Despite this, sign languages remain largely inaccessible to the hearing population, creating significant communication barriers in everyday life, healthcare, education, and public services. Automatic Sign Language Recognition (SLR) aims to bridge this gap by developing technological systems capable of understanding and interpreting sign language gestures automatically, thereby promoting inclusion and accessibility.

From a computational perspective, SLR is formulated as a sequence classification problem: given a temporal sequence of visual or skeletal observations, the goal is to predict the corresponding sign class. This task presents unique challenges stemming from the high intra-class variability introduced by signer-specific styles, viewpoint changes, and execution speed, as well as the nature of hand configurations that distinguish semantically different signs.

1.2 Motivation and Problem Statement

Recent advances in computer vision and deep learning have enabled significant progress in skeleton-based SLR, where human body keypoints extracted from video sequences serve as the primary input modality. Skeleton-based approaches offer several advantages [4, 8, 22]: they are compact, computationally efficient, robust to lighting and background variations, and preserve the privacy of signers by abstracting away appearance information..

Modern pose estimation tools such as MediaPipe Holistic [15] enable the extraction of 75 three-dimensional joint coordinates per frame, covering the upper body and both hands. These skeletal sequences capture the spatiotemporal dynamics of signing and form the basis of state-of-the-art recognition pipelines.

However, a fundamental challenge persists: raw joint coordinates do not explicitly encode the articulated kinematic structure of human motion or the rotational relationships between joints. Two signs may share a nearly identical hand shape yet differ entirely in meaning due to orientation — a distinction that Euclidean-based representations fail to capture reliably. Moreover, the large intra-class variability of signing styles causes coordinate-based models to overfit to motion amplitude rather than semantic intent.

The recent work of Sartinis et al. [19] addresses these limitations by proposing a geometry-aware framework for skeleton-based SLR. Their approach trains a Distance Encoder using complementary geometric distances — Euclidean (L2), Procrustes, and Articulated Pose Distance (APD) — via self-supervised triplet loss. The resulting embeddings are fused with raw skeletal features and provided as input to a Transformer-based recognition model, achieving substantial improvements on the WLASL-100 and WLASL-300 benchmarks [12].

Despite these gains, the triplet loss used in their pretraining stage is inherently self-supervised: it organizes the embedding space based on geometric similarity between poses, without any direct supervision from sign class labels.

This self-supervised formulation, while elegant, introduces several limitations. First, because positive and negative pairs are defined purely by geometric distance, the resulting embedding space is optimized to preserve *geometric* similarity rather than *semantic* similarity. Two poses that are geometrically close may nonetheless belong to different signs, and conversely, two poses from the same sign may be geometrically distant due to signer variability or execution speed. The triplet loss has no mechanism to correct for this mismatch, since it never observes the sign labels. As a result, the pretrained embeddings are not necessarily aligned with the ultimate classification ob-

jective. Second, triplet-based training depends heavily on the strategy used to sample positive and negative pairs; poorly chosen triplets can slow convergence or lead to suboptimal embeddings, and tuning this sampling process introduces additional complexity. Third, the self-supervised objective treats the geometric distance as a fixed proxy for similarity, which cannot adapt to the discriminative structure of the specific recognition task.

These limitations define the research gap that this work aims to address: *can the pose embedding encoder be pretrained in a way that directly incorporates sign class information, so that the learned embeddings are aligned with the recognition objective, while retaining the benefits of a dedicated embedding pretraining stage?* In other words, rather than organizing the embedding space around geometric proximity alone, we seek to organize it around class membership, so that poses belonging to the same sign are drawn together and poses from different signs are pushed apart. This motivates the central contribution of this work: the replacement of the self-supervised triplet loss with the supervised Proxy Anchor Loss [9], which leverages sign class labels during encoder pretraining while preserving the two-stage structure of the original framework. The following section describes the proposed approach in detail.

1.3 Proposed Approach

This work proposes the use of Proxy Anchor Loss [9] as a supervised alternative to triplet loss for pretraining the Distance Encoder within the framework of [19]. Originally proposed for deep metric learning in image retrieval, Proxy Anchor Loss assigns a learnable proxy vector to each class and jointly optimizes the embedding space by pulling all samples of the same class toward their proxy while pushing samples of different classes apart. Compared to triplet loss, it enables faster convergence, eliminates the need for careful triplet sampling strategies, and produces embeddings that are directly informed by class semantics rather than geometric proximity alone.

We evaluate this approach across three subsets of the WLASL benchmark WLASL-100, WLASL-300, and the full WLASL-2000 systematically assessing the Proxy Anchor encoder both in isolation and in combination with the geometric encoders of [19] through weighted and concatenated fusion strategies. We further explore the Proxy Anchor encoder under frozen and fine-tuned settings, and adapt the Hypergraph Transformer (Hyperformer) [23] as an alternative backbone.

Experimental results demonstrate that Proxy Anchor pretraining consistently improves recognition performance, surpassing the best configuration

reported in [19] on both WLASL-100 and WLASL-300, and establishing new competitive results on these benchmarks. On the more challenging WLASL-2000 benchmark, the behaviour of the Proxy Anchor embedding reveals an interesting trend: while its standalone Top-1 performance decreases as the vocabulary grows, when fused with the geometric encoders it consistently improves the Top-5 and Top-10 accuracy over the geometric-only baseline (by up to 2.6 and 1.9 percentage points respectively). This indicates that, at large vocabulary sizes, the class-supervised Proxy Anchor embedding acts as a valuable complementary signal that helps the model rank the correct sign highly among a large number of candidate classes, even when it is not the single top prediction.

1.4 Report Structure

The remainder of this work is organized as follows.

Chapter 2 presents the theoretical background, covering the fundamentals of sign language recognition, skeleton-based representations and the MediaPipe framework, self-supervised and metric learning, the Proxy Anchor Loss, the Transformer architecture, and the Hypergraph Transformer. It also describes the WLASL dataset used throughout the experiments.

Chapter 3 reviews the state of the art in isolated sign language recognition, covering both appearance-based and skeleton-based methods, representative geometry-aware approaches, and the standard benchmarks used to evaluate them. It positions the present work within the broader research landscape.

Chapter 4 presents the geometric distance measures that form the foundation of the embedding framework: the Euclidean distance, the rotation-regularized Procrustes distance, and the articulated pose distance based on quaternions. It also discusses the complementary nature of these measures.

Chapter 5 describes the methodology in detail, including the data pre-processing pipeline, the Distance Encoder architecture and its pretraining with Proxy Anchor Loss, the encoder fusion strategies, the Transformer-based recognition model, and the adaptation of the Hypergraph Transformer for the WLASL dataset.

Chapter 6 presents the experimental setup and results. It reports the performance of all evaluated configurations on WLASL-100, WLASL-300, and WLASL-2000, provides a comparison with the baseline framework and with state-of-the-art methods, analyzes the effect of vocabulary size, and includes the Hyperformer results.

Chapter 7 summarizes the findings of this work, discusses the limitations of the proposed approach, and outlines directions for future research.

Chapter 2

Theoretical Background

2.1 Sign Language and the Recognition Problem

Sign languages are complete, natural languages used by Deaf and hard-of-hearing communities as their primary means of communication. Unlike spoken languages, which rely on the auditory-vocal channel, sign languages are visual-gestural systems that convey meaning through the coordinated use of hand shapes, hand movements, spatial positioning, facial expressions, and body posture. Each country has developed its own sign language with a distinct grammar, syntax, and vocabulary. For instance, American Sign Language (ASL), British Sign Language (BSL), and Greek Sign Language (GSL) are entirely independent linguistic systems, not derived from their spoken counterparts and mutually unintelligible from one another.

Sign languages exhibit all the properties of natural human languages: they possess rich phonological, morphological, syntactic, and pragmatic structure. The basic unit of a sign language is the *sign* — the equivalent of a word in spoken language — which is defined by a combination of handshape, movement, location relative to the body, and palm orientation. These parameters, known as the *phonological parameters* of a sign, interact simultaneously rather than sequentially, giving sign languages a distinct spatial grammar that differs fundamentally from spoken language grammar.

The global Deaf community comprises an estimated 70 million people, according to the World Federation of the Deaf. Despite this, communication barriers between Deaf signers and the hearing population remain prevalent in everyday settings including healthcare, education, legal services, and public administration. Communication barriers also exist *between* signers of different sign languages: since each country has developed its own sign language

with distinct grammar and vocabulary, a signer of one sign language (for example, Greek Sign Language) cannot readily communicate with a signer of another (such as American Sign Language), just as speakers of different spoken languages cannot understand one another. Automatic sign language recognition systems therefore have the potential not only to bridge the gap between Deaf and hearing individuals, but also to serve as a foundation for translation between different sign languages. While human interpreters play an essential role in bridging this gap, they are not always available. This motivates the development of automatic Sign Language Recognition (SLR) systems capable of interpreting sign language from visual input, thereby improving accessibility and social inclusion for Deaf individuals.

2.1.1 Isolated vs. Continuous Sign Language Recognition

Sign language recognition is broadly categorized into two tasks: *isolated* and *continuous* SLR. In isolated SLR, the goal is to classify a single sign from a video or skeleton sequence that contains exactly one sign. This is a well-defined classification problem and constitutes the focus of the present study. In continuous SLR, the system must segment and recognize a sequence of signs from a continuous video stream, which introduces additional challenges such as sign boundary detection and co-articulation between consecutive signs.

This work focuses on isolated SLR using the WLASL benchmark [12], which contains videos of American Sign Language signs collected from the internet. The recognition task is formulated as a multi-class classification problem: given a temporal sequence of skeletal observations extracted from a sign video, the goal is to predict the correct sign label from a vocabulary of C classes.

2.1.2 Challenges in Sign Language Recognition

Despite significant progress in recent years, automatic SLR remains a challenging problem due to several factors:

Intra-class variability. The same sign can be executed with considerable variation between different signers, reflecting differences in articulation style, signing speed, and regional dialect. Even the same signer may produce the same sign differently depending on context, neighboring signs, or expressive intent.

Inter-class similarity. Many signs share very similar hand configurations or movement trajectories but differ in subtle aspects of orientation,

location, or non-manual markers such as facial expressions. Distinguishing between such *minimal pairs* requires fine-grained and geometrically precise representations.



Figure 2.1: Two signs with identical handshape but different orientation and meaning: (left) the number “one” and (right) the personal pronoun “she/he”. Such minimal pairs highlight the importance of orientation-aware representations for sign language recognition.

Limited annotated data. Large-scale annotated sign language datasets are difficult and expensive to collect, requiring skilled signers, professional annotation, and careful quality control. Most publicly available datasets contain only a few hundred samples per class, making generalization across signers and conditions particularly challenging.

Complex spatiotemporal structure. A sign unfolds over time through the coordinated motion of multiple body parts, requiring models that can jointly capture spatial relationships between joints and temporal dynamics across frames. Simple coordinate-based representations often fail to adequately model the articulated kinematic structure of hand and arm motion.

Occlusions and pose estimation errors. During signing, the hands frequently overlap with each other or with the face and body, causing one hand to partially or fully occlude the other. Such occlusions lead to missing or inaccurate keypoint detections by the pose estimation system, introducing noise into the skeletal representation. Rapid hand motion can additionally cause motion blur, further degrading the reliability of the extracted joints. These pose estimation errors are particularly problematic for methods that depend on precise joint coordinates, as even a small number of corrupted frames can affect the recognition of a sign.

These challenges motivate the development of geometry-aware and rep-

resentation learning approaches that go beyond raw coordinate features, as explored in this work.

2.2 Skeleton-based Representations and Pose Estimation

Skeleton-based representations have emerged as a dominant modality for sign language recognition and human action understanding. Rather than processing raw RGB video frames, skeleton-based approaches extract the 3D coordinates of anatomical keypoints — joints such as wrists, knuckles, and fingertips — and use these sparse structural representations as the primary input to recognition models. This paradigm offers several advantages over appearance-based methods: skeleton sequences are compact and computationally efficient, invariant to lighting conditions and background clutter, robust to changes in clothing and appearance, and provide an interpretable representation of human motion that aligns naturally with the linguistic structure of sign language.

The extraction of skeleton keypoints from video is performed by pose estimation systems, which have advanced significantly in recent years due to progress in deep learning. Modern pose estimators can localize dozens of body joints in real time from monocular RGB video, making skeleton-based SLR accessible without the need for specialized depth sensors.

2.2.1 MediaPipe Holistic

In this work, skeletal features are extracted using **MediaPipe Holistic** [15], a real-time pose estimation framework developed by Google that simultaneously detects body pose, hand landmarks, and face landmarks from a single RGB video stream. MediaPipe Holistic is particularly well-suited for sign language recognition because it provides detailed hand keypoints in addition to upper-body joints, capturing the fine-grained hand configurations that are linguistically distinctive in signed communication.

The full MediaPipe Holistic skeleton used in this work consists of **75 joints** per frame, organized as follows:

- **33 body pose joints:** covering the upper body, arms, and lower body, providing global positional context for sign articulation.
- **21 left hand joints:** covering the wrist and all finger joints of the left hand, enabling detailed modeling of left-hand configurations.

- **21 right hand joints:** covering the wrist and all finger joints of the right hand.

Each joint is represented by a 3D coordinate vector $(x, y, z) \in \mathbb{R}^3$, yielding a feature vector of dimension $75 \times 3 = 225$ per frame. A full sign sequence is thus represented as a matrix $\mathbf{X} \in \mathbb{R}^{T \times 225}$, where $T = 80$ is the fixed number of frames after temporal normalization.

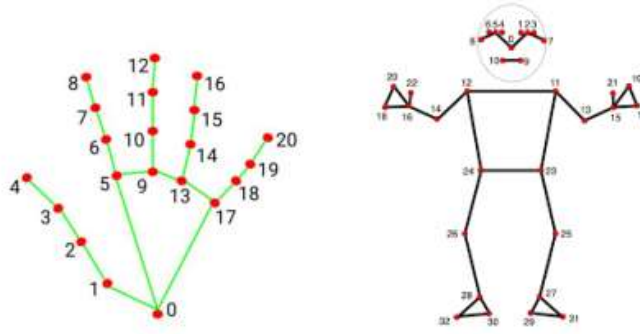


Figure 2.2: The MediaPipe Holistic skeleton consisting of 75 joints: 33 body pose joints, 21 left hand joints, and 21 right hand joints. Each joint is represented by a 3D coordinate (x, y, z) .

2.2.2 Skeletal Preprocessing

Raw skeletal sequences extracted by MediaPipe may vary in length due to differences in signing speed across signers and recordings. To ensure consistency across samples, all sequences are temporally normalized to a fixed length of $T = 80$ frames through linear interpolation. Joint coordinates are subsequently standardized independently per dimension using z-score normalization, removing global scale differences between recordings.

For the purposes of geometric embedding learning, the right and left hand joints are processed independently using fixed index slices of the full feature vector. The right hand occupies joints 54 through 74 (indices $[54, 75)$) and the left hand occupies joints 33 through 53 (indices $[33, 54)$) in the 75-joint skeleton. This separation allows geometry-aware encoders to focus on the detailed articulated structure of each hand independently, which is particularly relevant given that many signs are distinguished by right-hand configuration alone.

2.2.3 Advantages and Limitations

The use of MediaPipe Holistic [15] for skeleton extraction offers practical advantages: it is freely available, runs in real time on standard hardware without a GPU, and provides consistent joint localization across diverse recording conditions. However, skeleton-based representations also have inherent limitations. Occlusion of one hand by the other, rapid motion blur, or unusual camera angles can cause keypoint detection failures or inaccuracies. Furthermore, non-manual markers such as facial expressions and mouth patterns — which carry grammatical and lexical information in sign language — are not captured by the 75-joint skeleton used in this work, potentially limiting recognition performance for signs that are distinguished primarily by facial features.

Despite these limitations, skeleton-based representations remain a powerful and widely adopted modality for sign language recognition, and the experiments reported in this work demonstrate that geometry-aware augmentation of these representations can substantially improve recognition accuracy.

2.3 Self-Supervised and Metric Learning

A central challenge in sign language recognition is the scarcity of large annotated datasets. Collecting and labeling sign language videos requires skilled signers and expert annotators, making fully supervised learning difficult to scale. Self-supervised and metric learning approaches address this by learning meaningful representations from data with minimal or no manual annotation, relying instead on structural properties of the data itself to define learning objectives.

2.3.1 Representation Learning and Embedding Spaces

The goal of representation learning is to map raw input data — in our case, skeletal pose sequences — into a compact latent space where semantically similar inputs are geometrically close and dissimilar inputs are far apart. A learned embedding function $g_\phi : \mathbb{R}^{J \times 3} \rightarrow \mathbb{R}^d$ maps each pose into a d -dimensional embedding vector, where ϕ denotes the learnable parameters of the network.

The quality of the learned embedding space is determined by the loss function used during training. Most metric learning losses can be categorized into two broad classes: *pair-based* losses and *proxy-based* losses, each with distinct advantages and limitations.

2.3.2 Pair-based Losses

Pair-based losses are built upon pairwise or tuple-wise distances between data points in the embedding space. They provide rich supervisory signals by directly comparing embedding vectors of different samples.

Contrastive Loss [5] is one of the earliest metric learning objectives. It takes pairs of samples and minimizes the distance between embeddings of the same class while maximizing the distance between embeddings of different classes:

$$\mathcal{L}_{\text{contrastive}} = y \cdot d(x_i, x_j)^2 + (1 - y) \cdot \max(0, m - d(x_i, x_j))^2 \quad (2.1)$$

where $y \in \{0, 1\}$ indicates whether the pair shares the same label and m is a margin hyperparameter.

Triplet Loss [20] extends this idea by considering triplets of samples: an anchor x_a , a positive sample x_p of the same class, and a negative sample x_n of a different class. The loss enforces that the anchor is closer to the positive than to the negative by a margin α :

$$\mathcal{L}_{\text{triplet}} = \max(0, \|g(x_a) - g(x_p)\|_2^2 - \|g(x_a) - g(x_n)\|_2^2 + \alpha) \quad (2.2)$$

Triplet loss has been widely adopted in self-supervised embedding learning for skeleton-based recognition [6], including in the geometric embedding framework of [19], where triplets are constructed based on geometric distance rather than class labels. This allows the embedding network to preserve the relative ordering induced by the geometric distance function without requiring any class annotations.

Despite their effectiveness, pair-based losses suffer from high training complexity — $O(M^2)$ for contrastive loss and $O(M^3)$ for triplet loss, where M is the number of training samples — and require careful triplet sampling strategies to avoid uninformative or degenerate tuples that slow convergence.

2.3.3 Proxy-based Losses

Proxy-based losses address the complexity issue of pair-based methods by introducing proxies learnable representative vectors, typically one per class that serve as anchors in the embedding space. Instead of comparing data points to each other, each sample is compared to proxies, reducing training complexity to $O(MC)$ where M is the number of samples and C the number of classes. Since the number of proxies is much smaller than the number of

possible data pairs, proxy-based losses converge faster and are more robust to noisy labels and outliers.

Proxy-NCA [16] is the seminal proxy-based loss. It assigns a single proxy to each class and, given a data point as an anchor, encourages it to be close to the proxy of its own class and far from all other proxies:

$$\mathcal{L}_{\text{NCA}}(x) = -\log \frac{e^{s(x, p^+)}}{\sum_{p^- \in \mathcal{P}^-} e^{s(x, p^-)}} \quad (2.3)$$

where $s(\cdot, \cdot)$ denotes cosine similarity, p^+ is the positive proxy and $\mathcal{P}^-$ is the set of negative proxies. While Proxy-NCA enables fast convergence, it associates each data point only with proxies, losing the rich data-to-data relations available in pair-based losses.

SoftTriple [18] extends Proxy-NCA by assigning multiple proxies per class rather than a single one. This allows the loss to capture intra-class variance for example, different ways of executing the same sign, which a single proxy per class cannot represent. The trade-off is an increased number of proxies and a corresponding rise in training complexity.

Manifold Proxy loss [3] extends the N-pair loss using proxies and adopts a manifold-aware distance instead of the Euclidean distance to measure semantic similarity in the embedding space, aiming to better reflect the underlying structure of the data.

A common limitation of these proxy-based losses is that, because each data point is associated only with proxies and not with other data points, they cannot fully exploit the fine-grained data-to-data relations that pair-based losses leverage. The Proxy Anchor Loss [9], described in the following section, was proposed specifically to overcome this limitation while retaining the fast convergence and robustness of proxy-based methods.

2.3.4 Self-Supervised Embedding Learning

In the self-supervised setting, the learning objective is defined without using class labels. Instead, positive and negative pairs are constructed based on structural properties of the data. In the framework of [19], triplets are constructed based on geometric distance: poses that are geometrically similar (according to L2, Procrustes, or APD) are treated as positives, while dissimilar poses are treated as negatives. This allows the embedding network to learn a geometrically consistent latent space that captures articulated pose structure without any supervision from sign class labels.

The key advantage of self-supervised embedding learning in this context is that it can exploit the rich geometric structure of unlabeled pose data, pro-

viding strong inductive biases for the downstream recognition model. The pretrained encoder is subsequently integrated into the Transformer-based recognition model and fine-tuned jointly with the full network during classification training.

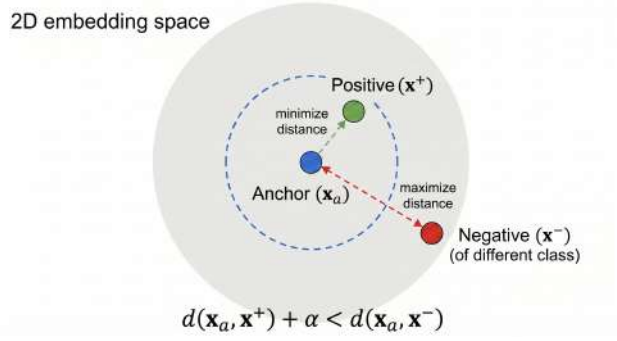


Figure 2.3: Illustration of the triplet loss objective. The loss encourages the anchor embedding to be closer to the positive (same class or geometrically similar) than to the negative (different class or geometrically dissimilar) by a margin α .

2.4 Proxy Anchor Loss

Proxy Anchor Loss [9] is a metric learning objective that combines the advantages of both pair-based and proxy-based approaches while overcoming their respective limitations. It was originally proposed for deep metric learning in image retrieval tasks, achieving state-of-the-art performance on standard benchmarks while converging significantly faster than pair-based alternatives. In this work, we adopt Proxy Anchor Loss as a supervised alternative to triplet loss for pretraining the Distance Encoder in the sign language recognition framework.

2.4.1 Core Idea

The key insight of Proxy Anchor Loss is to use each proxy as an *anchor* rather than each data point. Unlike Proxy-NCA, which associates each data point with proxies, Proxy Anchor Loss associates each proxy with all data points in a batch simultaneously. This design enables the loss to leverage rich data-to-data relations through its gradients: the gradient with respect to any data point depends on the relative hardness of all other data points in the batch, allowing the embedding network to learn from fine-grained

inter-sample relationships without the combinatorial complexity of triplet sampling.

Formally, a single proxy p is assigned to each class following the standard proxy assignment of Proxy-NCA [16]. Proxies are learnable parameters initialized from a normal distribution and updated jointly with the embedding network during training. For each proxy p , the batch of embedding vectors $\mathcal{X}$ is divided into two sets: $\mathcal{X}_p^+$, the set of positive embeddings belonging to the same class as p , and $\mathcal{X}_p^- = \mathcal{X} \setminus \mathcal{X}_p^+$, the set of negative embeddings.

2.4.2 Loss Formulation

The Proxy Anchor Loss is defined as:

$$\mathcal{L} = \frac{1}{|\mathcal{P}^+|} \sum_{p \in \mathcal{P}^+} \log \left(1 + \sum_{x \in \mathcal{X}_p^+} e^{-\alpha(s(x,p)-\delta)} \right) + \frac{1}{|\mathcal{P}|} \sum_{p \in \mathcal{P}} \log \left(1 + \sum_{x \in \mathcal{X}_p^-} e^{\alpha(s(x,p)+\delta)} \right) \quad (2.4)$$

where $s(x, p)$ denotes the cosine similarity between embedding x and proxy p , $\delta > 0$ is a margin, $\alpha > 0$ is a scaling factor, $\mathcal{P}$ is the set of all proxies, and $\mathcal{P}^+$ is the set of proxies whose class is represented by at least one sample in the current batch.

The first term in Equation 2.4 is the *positive term*: it encourages each proxy to pull all embeddings of the same class closer than the margin δ . The second term is the *negative term*: it pushes all embeddings of different classes beyond the margin. Both terms use the Log-Sum-Exp function, which acts as a smooth approximation of the maximum, effectively focusing the gradient on the hardest positive and negative examples within the batch while still considering all examples through soft weighting.

The gradient of the loss with respect to the cosine similarity $s(x, p)$ is:

$$\frac{\partial \mathcal{L}}{\partial s(x, p)} = \begin{cases} \frac{-\alpha h_p^+(x)}{|\mathcal{P}^+| \left(1 + \sum_{x' \in \mathcal{X}_p^+} h_p^+(x') \right)}, & x \in \mathcal{X}_p^+ \\ \frac{\alpha h_p^-(x)}{|\mathcal{P}| \left(1 + \sum_{x' \in \mathcal{X}_p^-} h_p^-(x') \right)}, & x \in \mathcal{X}_p^- \end{cases} \quad (2.5)$$

where $h_p^+(x) = e^{-\alpha(s(x,p)-\delta)}$ and $h_p^-(x) = e^{\alpha(s(x,p)+\delta)}$ are the positive and negative hardness metrics respectively. A positive embedding that is far from its proxy yields a large $h_p^+(x)$, receiving a stronger gradient pull. Similarly, a negative embedding that is close to a proxy yields a large $h_p^-(x)$, receiving a stronger gradient push. This relative hardness weighting determined by

all embeddings in the batch, is what allows Proxy Anchor Loss to leverage data-to-data relations despite being proxy-based.

Behaviour of the Loss During Training

To illustrate how the two terms of the Proxy Anchor Loss behave during optimization, Figure 2.4 shows the evolution of the positive term, the negative term, and the total loss on a minimal synthetic example. The example consists of three classes with thirty two-dimensional ℓ_2 -normalized embeddings each, initialized randomly on the unit circle together with one learnable proxy per class, and optimized by gradient descent using the Proxy Anchor objective with $\alpha = 32$ and $\delta = 0.1$.

At the beginning of training, both terms are large: embeddings are far from their own proxies (large positive term) and close to wrong proxies (large negative term). As training progresses, the positive term decreases as each proxy pulls the embeddings of its class within the margin, while the negative term decreases as embeddings of other classes are pushed away. The total loss decreases smoothly and converges once the classes are well separated in the embedding space.

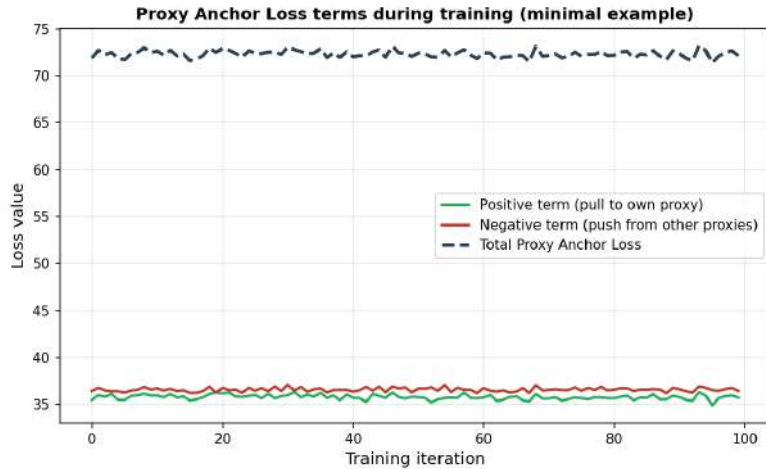


Figure 2.4: Evolution of the positive term, negative term, and total Proxy Anchor Loss during training on a minimal synthetic example (three classes, two-dimensional embeddings on the unit circle, $\alpha = 32$, $\delta = 0.1$). Both terms decrease as embeddings are pulled toward their own proxy and pushed away from the others.

Gradient Behaviour and Hardness Weighting

The key property of the Proxy Anchor Loss lies in its gradients. Figure 2.5 visualizes the gradient magnitude as a function of the cosine similarity between an embedding and a proxy, for positive and negative examples. For positive examples (left), the pull strength is large when the embedding is far from its own proxy (a *hard positive*) and small when it is already close (an *easy positive*). For negative examples (right), the push strength is large when the embedding is close to a wrong proxy (a *hard negative*). In other words, the loss automatically focuses its gradients on the most informative examples, without any explicit sampling procedure.

The figure also contrasts this behaviour with Proxy-NCA: as shown in Equation (2.x) of Section 2.3.3, the gradient of Proxy-NCA with respect to a positive example has a *constant* scale regardless of how far the embedding is from its proxy. Proxy Anchor instead modulates the gradient scale according to the relative hardness of each example within the batch, which has been shown to improve the flexibility and generalization of the learned embedding space [9].

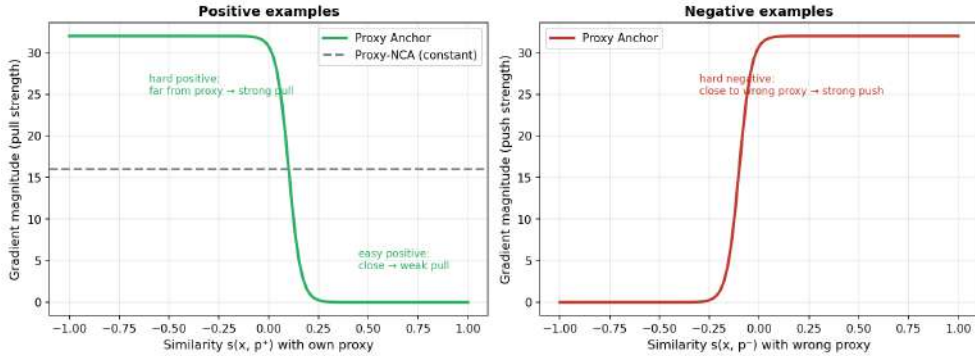


Figure 2.5: Gradient magnitude of the Proxy Anchor Loss as a function of the similarity between an embedding and a proxy ($\alpha = 32$, $\delta = 0.1$). Left: for positive examples, hard positives (far from their proxy) receive a strong pull, while easy positives receive a weak one; Proxy-NCA, in contrast, applies a constant-scale gradient to all positives. Right: for negative examples, hard negatives (close to a wrong proxy) receive a strong push.

Comparison with Proxy-NCA on a Minimal Example

To make the difference from the mainstream proxy-based loss concrete, the same minimal synthetic problem was trained with both Proxy-NCA and Proxy Anchor, starting from an identical initialization. Figure 2.6 shows

the final embedding spaces produced by the two losses and their convergence curves.

Both losses successfully separate the three classes. However, the Proxy Anchor embedding space exhibits visibly tighter intra-class clusters around each proxy and larger inter-class margins, a direct consequence of the margin δ and the hardness-aware gradients: hard positives are pulled more strongly toward their proxy, and hard negatives are pushed more strongly away. The convergence plot further shows that Proxy Anchor reaches high nearest-proxy accuracy in fewer iterations, consistent with the faster convergence reported in the original work [9]. While this is a deliberately simple two-dimensional example, it captures the essential mechanism that motivates the use of Proxy Anchor Loss for the encoder pretraining in this work.

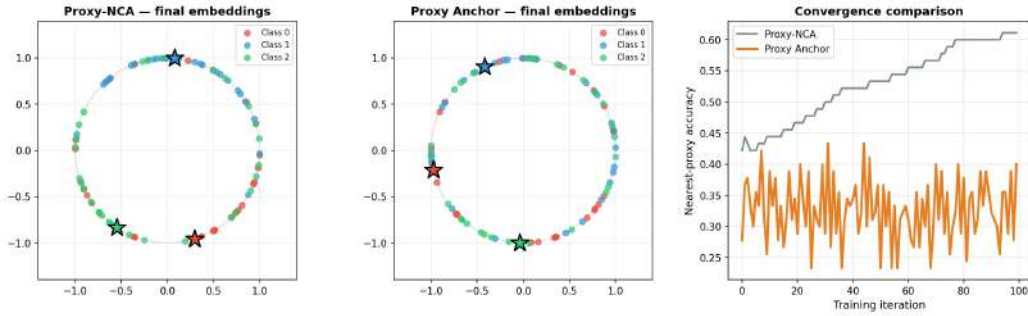


Figure 2.6: Minimal-example comparison between Proxy-NCA and Proxy Anchor. Left and middle: final two-dimensional embedding spaces produced by each loss from the same initialization (stars denote the learned proxies). Proxy Anchor produces tighter intra-class clusters and larger inter-class margins. Right: nearest-proxy accuracy during training, showing the faster convergence of Proxy Anchor.

2.4.3 Hyperparameters

Following the original paper [9], the scaling factor and margin are set to $\alpha = 32$ and $\delta = 0.1$ respectively for all experiments in this work. The scaling factor α controls the sharpness of the hardness weighting: larger values concentrate gradients more strongly on hard examples. The margin δ enforces a separation between positive and negative embeddings in cosine similarity space. Sensitivity analysis reported in [9] shows that the loss is robust to these hyperparameter choices for $\alpha \geq 16$.

2.4.4 Advantages over Triplet Loss

Proxy Anchor Loss offers several practical advantages over triplet loss for the pretraining of the Distance Encoder in our framework:

No triplet sampling required. Triplet loss requires the careful construction of informative triplets, which typically involves hard or semi-hard negative mining strategies that introduce additional hyperparameters and computational overhead. Proxy Anchor Loss eliminates this requirement entirely, as every embedding in the batch contributes to the loss through its relationship with all proxies.

Supervised signal. While the triplet loss used in [19] is self-supervised — relying on geometric distances to define positive and negative pairs — Proxy Anchor Loss directly incorporates sign class labels. This provides a stronger and more semantically meaningful training signal, producing embeddings that are explicitly optimized for the downstream classification task.

Training complexity. The training complexity of Proxy Anchor Loss is $O(MC)$, where M is the number of training samples and C is the number of classes. This is the same as Proxy-NCA and substantially lower than the $O(M^3)$ complexity of naive triplet loss, enabling efficient training even for large vocabularies such as WLASL-300.

Faster convergence. Due to the proxy-based design and the absence of sampling strategies, Proxy Anchor Loss converges more reliably and in fewer epochs compared to pair-based losses, as demonstrated empirically in [9].

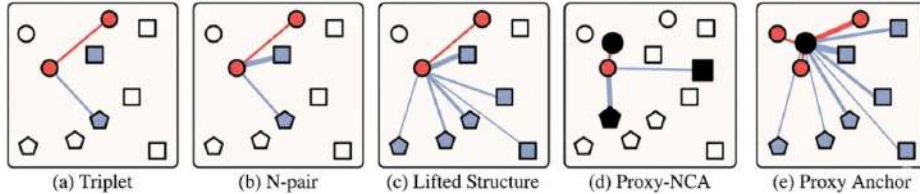


Figure 2.7: Comparison between popular metric learning losses and Proxy Anchor Loss. Small nodes represent embedding vectors and black nodes represent proxies. Thicker edges indicate larger gradients. Unlike pair-based losses (Triplet, N-pair) and Proxy-NCA, Proxy Anchor Loss associates each proxy with all data in the batch, enabling rich data-to-data interactions through gradient weighting. Figure adapted from [9].

2.4.5 Limitations of Proxy Anchor Loss

Despite its advantages, the Proxy Anchor Loss also has limitations that should be acknowledged, particularly in the context of sign language recog-

dition.

Requirement for labeled data. The most fundamental limitation is that, unlike the self-supervised triplet formulation of [19], Proxy Anchor Loss requires class labels for every training sample. This means it cannot exploit unlabeled pose data, which is typically far more abundant than annotated data. In scenarios where annotations are scarce or expensive — a common situation in sign language research — the self-supervised alternative retains an important practical advantage.

Scalability with the number of classes. The loss maintains one learnable proxy per class, so the number of proxy parameters grows linearly with the vocabulary size. More importantly, as the number of classes increases, each proxy must discriminate its class against an ever-growing set of negatives, and individual frames become increasingly ambiguous across classes. This effect was observed directly in the experiments of this work: the standalone performance of the Proxy Anchor encoder declines relative to the geometric encoders as the vocabulary grows from 100 to 2,000 classes.

Frame-level label noise. In our setting, the loss is applied at the frame level, assigning to every frame the label of the sign it belongs to. However, not all frames of a sign are equally informative: frames at the onset, offset, or during transitional movements may be nearly identical across different signs, effectively introducing label noise into the pretraining. The triplet formulation, being based on geometric similarity rather than labels, is unaffected by this issue.

Sensitivity to class imbalance. Since each proxy is updated by the samples of its own class, classes with few training samples receive fewer updates to their proxies, which may result in less reliable proxy positions for rare signs. Datasets such as WLASL, where the number of samples per class varies considerably, can therefore pose challenges for proxy-based training.

Single proxy per class. Following the standard setting, a single proxy is assigned to each class. This implicitly assumes that each class forms a single compact cluster in the embedding space. Signs with high intra-class variability — for example, due to regional variants or signer-specific styles — may be better represented by multiple proxies per class, as in SoftTriple [18], at the cost of additional parameters and complexity.

These limitations do not negate the benefits demonstrated in this work, but they delineate the conditions under which the Proxy Anchor objective is most effective: moderate vocabulary sizes, sufficient labeled samples per class, and settings where it can be combined with complementary representations — precisely the configuration explored in the following chapters.

2.5 Transformer Architecture for Sequence Modeling

The Transformer [21] is a neural network architecture originally proposed for natural language processing that has since become the dominant model for sequence modeling across a wide range of domains, including computer vision, speech recognition, and human action understanding. Its success is largely attributed to the *self-attention* mechanism, which allows the model to capture long-range dependencies between elements of a sequence without the sequential processing constraints of recurrent architectures such as LSTMs and GRUs.

In the context of sign language recognition, Transformer encoders are particularly well-suited for modeling the temporal dynamics of skeletal sequences, as they can jointly attend to any pair of frames in the input sequence regardless of their temporal distance. The framework of [19] adopts a Transformer encoder as the backbone for sign recognition, and this architecture forms the foundation of the models evaluated in this work.

2.5.1 Self-Attention Mechanism

The core building block of the Transformer is the *scaled dot-product attention* mechanism. Given an input sequence of feature vectors, three linear projections are applied to produce queries Q , keys K , and values V :

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V \quad (2.6)$$

where d_k is the dimensionality of the key vectors. The softmax operation produces attention weights that reflect the relevance of each position to every other position in the sequence. The $\sqrt{d_k}$ scaling factor prevents the dot products from growing too large in magnitude, which would push the softmax into regions with very small gradients.

Multi-Head Attention extends this by running h attention heads in parallel, each with its own learned projection matrices:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O \quad (2.7)$$

where $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$. Each head can attend to different aspects of the input representation, allowing the model to jointly reason over multiple complementary patterns in the sequence.

2.5.2 Transformer Encoder

The Transformer encoder consists of a stack of identical layers, each containing two sub-layers: a multi-head self-attention module and a position-wise feed-forward network (FFN). Residual connections and layer normalization are applied around each sub-layer:

$$\mathbf{h} = \text{LayerNorm}(\mathbf{x} + \text{MultiHead}(\mathbf{x})) \quad (2.8)$$

$$\mathbf{z} = \text{LayerNorm}(\mathbf{h} + \text{FFN}(\mathbf{h})) \quad (2.9)$$

The feed-forward network applies two linear transformations with a ReLU activation in between, independently at each position. This combination of attention and feed-forward layers enables the encoder to learn rich contextual representations of each frame in the sequence.

2.5.3 Positional Encoding

Since the self-attention mechanism is permutation-invariant — it treats the input as a set rather than a sequence — positional information must be explicitly injected into the input representations. This is done through *positional encodings*, which are fixed or learnable vectors added to the input embeddings:

$$\text{PE}(\text{pos}, 2i) = \sin\left(\frac{\text{pos}}{10000^{2i/d}}\right), \quad \text{PE}(\text{pos}, 2i + 1) = \cos\left(\frac{\text{pos}}{10000^{2i/d}}\right) \quad (2.10)$$

where pos is the position in the sequence and i is the dimension index. These sinusoidal encodings allow the model to distinguish between frames at different temporal positions and generalize to sequence lengths not seen during training.

2.5.4 Transformer for Sign Language Recognition

In the sign recognition framework used in this work [19], the Transformer encoder processes a sequence of per-frame feature tokens. At each time step t , the geometry-aware embeddings produced by the pretrained Distance Encoder are concatenated with the raw joint coordinate features, forming an augmented input token. These tokens are then passed through the Transformer encoder, which models temporal dependencies across all frames through self-attention.

Final predictions are obtained by applying temporal average pooling over the output representations of all frames, followed by a linear classification

head that maps the pooled representation to the sign class probabilities. The full architecture is trained end-to-end using cross-entropy loss, with the pretrained Distance Encoder fine-tuned jointly with the Transformer.

The Transformer is configured with 4 layers, a hidden dimension of 256, and 8 attention heads. The AdamW optimizer [14] is used with a learning rate of 3×10^{-4} and weight decay 10^{-4} , with a cosine annealing learning rate schedule over 50 training epochs.

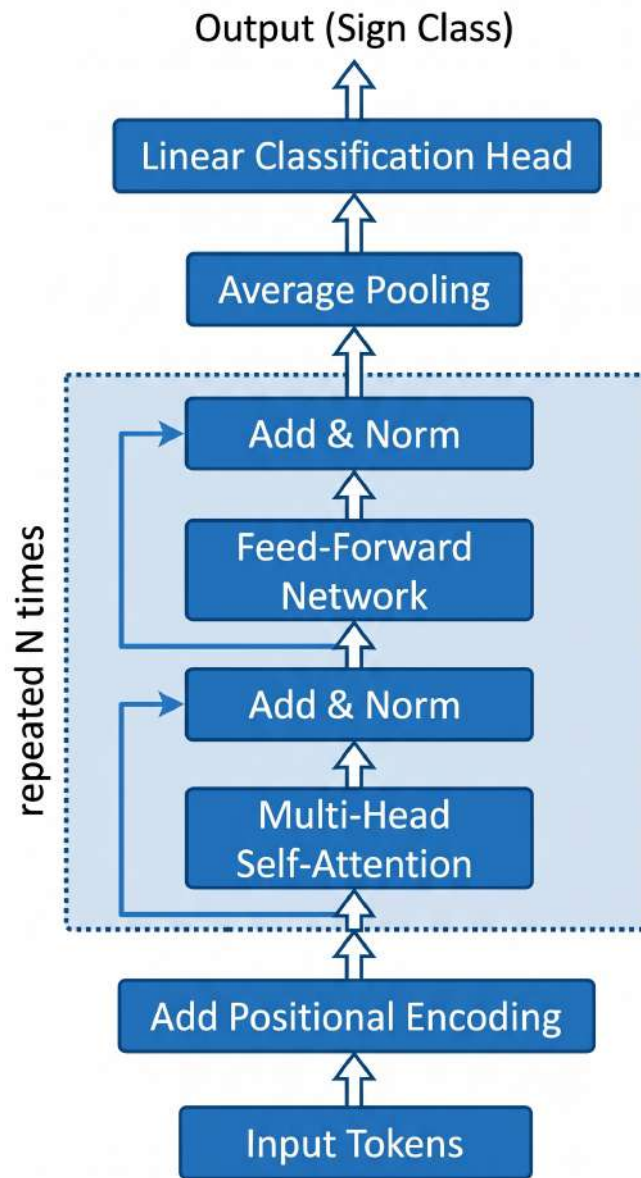


Figure 2.8: Overview of the Transformer encoder architecture used for sign language recognition. Input tokens combining geometry-aware embeddings and raw joint features are processed through stacked self-attention and feed-forward layers, followed by temporal average pooling and a linear classification head.

2.6 Hypergraph Transformer for Skeleton-based Action Recognition

The Hypergraph Transformer (Hyperformer) [23] is a skeleton-based action recognition model that combines hypergraph-based representations of skeletal topology with the self-attention mechanism of Transformers. It is evaluated in this work as an alternative backbone to the vanilla Transformer, adapted for MediaPipe skeletal data and assessed both with standard cross-entropy loss and with Proxy Anchor Loss.

2.6.1 From Graphs to Hypergraphs

To understand the contribution of the Hyperformer, it is useful to first recall how conventional graph-based methods model the skeleton. In a standard Graph Convolutional Network (GCN) such as ST-GCN [22], the skeleton is represented as a graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where the vertices $\mathcal{V}$ correspond to the joints and the edges $\mathcal{E}$ connect pairs of anatomically adjacent joints (for example, the wrist and the elbow). Each edge therefore encodes a *pairwise* relationship between exactly two joints. Graph convolution then aggregates information for each joint by averaging or weighting the features of its directly connected neighbours.

This pairwise formulation has an inherent limitation: it can only represent relationships between two joints at a time. Many meaningful patterns in human motion, however, involve the coordinated movement of *groups* of joints that are not necessarily adjacent in the skeleton. For example, in sign language, a particular handshape is defined by the joint configuration of all five fingers simultaneously, and the relationship between the two wrists and the torso conveys important spatial information. A standard graph cannot directly encode such higher-order, group-level relationships; it can only approximate them indirectly through multiple layers of pairwise message passing.

A *hypergraph* $\mathcal{H} = (\mathcal{V}, \mathcal{E}_h)$ generalizes the graph by allowing each *hyperedge* $e \in \mathcal{E}_h$ to connect an arbitrary number of vertices, rather than just two. A single hyperedge can therefore group together, for instance, all the joints of one hand, or a set of joints that move together during a specific type of motion. This allows the model to represent higher-order dependencies explicitly and in a single operation, rather than approximating them through repeated pairwise interactions. Figure 2.9 illustrates the difference between a standard graph and a hypergraph representation of the skeleton.

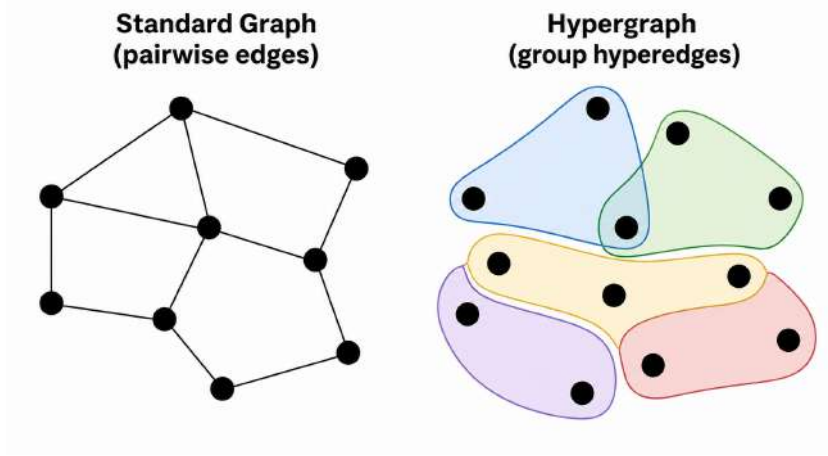


Figure 2.9: Comparison between a standard graph (left) and a hypergraph (right) representation of a skeleton. In the standard graph, each edge connects exactly two joints, capturing only pairwise relationships. In the hypergraph, a single hyperedge (shaded region) can connect multiple joints simultaneously, capturing higher-order group relationships such as the joint configuration of all fingers of hand.

2.6.2 Key Differences from Standard Graph Convolution

The Hyperformer differs from a conventional GCN-based model in three main respects.

Higher-order connectivity. As described above, the most fundamental difference is the use of hyperedges that connect groups of joints rather than pairs. This allows the Hyperformer to capture coordinated multi-joint patterns directly, which is particularly relevant for the fine-grained hand configurations that distinguish signs.

Attention-based aggregation instead of fixed graph convolution. In a standard GCN, the aggregation of neighbouring joint features is governed by a fixed adjacency matrix derived from the skeleton topology. The Hyperformer instead employs self-attention over the joints, allowing the model to learn data-dependent relationships between joints rather than relying solely on the predefined skeleton structure. The attention weights are additionally informed by a joint grouping that assigns each joint to a semantic group, so that the model learns group-aware attention patterns. This combines the structural prior of the skeleton with the flexibility of learned attention.

Joint spatial-temporal modeling. The Hyperformer integrates spatial joint modeling and temporal modeling within a unified architecture. Each

building block combines a spatial self-attention module operating over the joints with a multi-scale temporal convolutional module operating over the time dimension, allowing the model to jointly reason about which joints are related and how their relationships evolve over time.

2.6.3 Hyperformer Architecture

The Hyperformer consists of a stack of building blocks, each combining a spatial self-attention module with a multi-scale temporal convolutional network (MS-TCN). The spatial module applies multi-head self-attention over the joint dimension at each time step, conditioned on the joint grouping labels, to model relationships between joints. The temporal module applies convolutions with multiple kernel sizes and dilation rates over the time dimension to capture motion patterns at different temporal scales.

The model used in this work consists of 10 such blocks with a channel dimension of 144. Two downsampling layers reduce the temporal resolution by a factor of two each, compressing the sequence from $T = 80$ to $T = 20$ frames. Temporal average pooling followed by a linear classification head produces the final sign class predictions.

2.6.4 Joint Grouping for MediaPipe

The original Hyperformer was designed for the NTU RGB+D dataset [13] with a 25-joint body skeleton. In this work, it is adapted for the MediaPipe Holistic skeleton with 75 joints by defining a custom joint grouping that assigns each joint to one of eight semantic groups: body joints, left and right arm joints, left and right wrist regions, left and right hand palm joints, and finger joints. This grouping reflects the anatomical structure of the skeleton and enables the spatial attention module to learn group-aware relationships between semantically related joints.

2.6.5 Limitations for Sign Language Recognition

While the Hyperformer achieves strong results on large-scale action recognition datasets such as NTU RGB+D (which contains over 56,000 samples), its application to sign language recognition on WLASL presents significant challenges. The WLASL-100 training set contains only 1,408 samples — approximately 40 times fewer than NTU RGB+D — which is insufficient for the Hyperformer to fully learn the complex hypergraph attention patterns it was designed for. This data scarcity, combined with the hardware constraints of the available GPU, which was insufficient to train the full model locally

and required the use of Google Colab, explains the lower performance of the Hyperformer compared to the vanilla Transformer on WLASL, as discussed in Chapter 6.

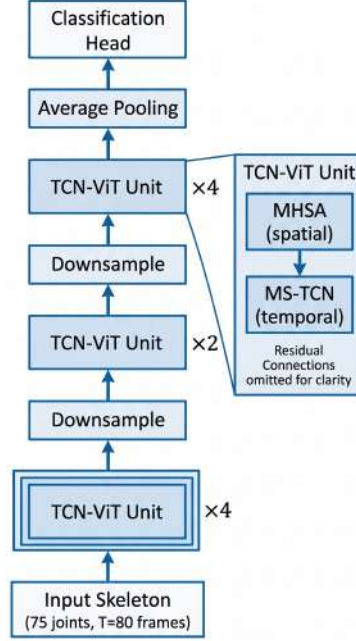


Figure 2.10: Overview of the Hypergraph Transformer (Hyperformer) architecture adapted for MediaPipe skeletal data. Each TCN-ViT unit combines spatial multi-head self-attention (MHSA) over joints with multi-scale temporal convolutions (MS-TCN) over frames. The model consists of 10 such units with two temporal downsampling layers.

2.7 The WLASL Dataset

The Word-Level American Sign Language (WLASL) dataset [12] is one of the largest publicly available datasets for isolated sign language recognition. It was introduced to address the lack of large-scale, diverse, and publicly accessible benchmarks for American Sign Language recognition, providing a challenging testbed for evaluating modern deep learning approaches.

2.7.1 Dataset Description

WLASL contains videos of American Sign Language signs collected from publicly available online sources, including educational websites and Deaf

community platforms. The dataset covers a vocabulary of 2,000 ASL glosses performed by over 100 different signers, making it one of the most diverse sign language datasets in terms of both vocabulary size and signer variability. The large number of signers introduces significant intra-class variation, as different individuals produce the same sign with distinct articulation styles, speeds, and spatial extents.

The dataset is organized into four subsets of increasing vocabulary size and difficulty: WLASL-100, WLASL-300, WLASL-1000, and WLASL-2000, containing the 100, 300, 1000, and 2000 most frequent glosses respectively. In this work, experiments are conducted on the two smallest subsets, WLASL-100 and WLASL-300, following the evaluation protocol of [19].

2.7.2 Data Splits

The dataset statistics for the two subsets used in this work are summarized in Table 2.1. Each subset is divided into training, validation, and test splits. The skeletal features used in our experiments were preprocessed and provided as `.npy` files, with each sample represented as a matrix of shape $(T, 225)$ where $T = 80$ is the fixed sequence length and $225 = 75 \times 3$ corresponds to the 75 MediaPipe joints with 3D coordinates.

Table 2.1: Dataset statistics for the WLASL subsets used in this work.

Subset	Classes	Train	Validation	Test
WLASL-100	100	1,408	329	249
WLASL-300	300	3,459	887	648

2.7.3 Challenges

WLASL presents several challenges that make it a demanding benchmark for sign language recognition:

Signer variability. With over 100 different signers, the dataset captures a wide range of articulation styles. Models must generalize across signers rather than memorizing individual signing habits.

Limited samples per class. On average, WLASL-100 contains approximately 14 training samples per class and WLASL-300 approximately 12. This severe data scarcity makes it difficult to train large models from scratch and motivates the use of pretraining and embedding-based approaches.

Noisy data. Since the videos were collected from the internet rather than controlled recording conditions, some samples contain inconsistencies

in signing style, partial occlusions, or low video quality that can affect the quality of the extracted skeletal features.

Class imbalance. The number of samples per class varies considerably across the dataset, as more frequent glosses tend to have more available video recordings than rare ones.

2.7.4 Evaluation Protocol

Following the standard evaluation protocol for WLASL [12] and the experimental setup of [19], recognition performance is reported using Top-1, Top-5, and Top-10 accuracy metrics on the test split. Top- k accuracy measures the proportion of test samples for which the correct class label appears among the k highest-scoring predictions of the model. Top-5 and Top-10 accuracy provide a more lenient measure of performance that is particularly informative for large-vocabulary recognition tasks where the top prediction may be a semantically related sign.



Figure 2.11: Example frames from the WLASL dataset showing different signers performing ASL signs. The large inter-signer variability in articulation style, speed, and spatial extent is a key challenge for recognition models.

Chapter 3

State of the Art

This chapter reviews the current state of the art in isolated sign language recognition, focusing on the two dominant input modalities: appearance-based methods that operate directly on RGB video, and skeleton-based methods that operate on extracted body and hand keypoints. We also review the standard benchmarks used to evaluate these methods and position the present work within the broader research landscape.

3.1 Isolated Sign Language Recognition

Isolated sign language recognition aims to classify a single sign from a video clip that contains exactly one gloss. It serves as a fundamental building block for the more general task of continuous sign language recognition and translation. Over the past decade, research in isolated SLR has progressed from early statistical models toward deep learning architectures, driven largely by the availability of larger datasets and more powerful computational resources.

Early approaches relied on Hidden Markov Models (HMMs) combined with handcrafted features. For instance, depth sensors and Kinect skeletons paired with HMMs achieved high accuracy on small vocabularies of a few dozen signs [11]. However, these methods scaled poorly to larger vocabularies and were sensitive to variations in signing style and recording conditions. The introduction of deep learning shifted the field toward data-driven feature learning, with convolutional and recurrent architectures replacing handcrafted pipelines.

3.2 Appearance-based Methods

Appearance-based methods operate directly on RGB (and sometimes depth) video frames, learning spatiotemporal features that capture both the visual appearance and the motion of the signer. These methods typically employ 2D or 3D convolutional neural networks (CNNs) to extract per-frame or per-clip features, often combined with recurrent or attention-based modules to model temporal dynamics.

3D CNNs such as I3D (Inflated 3D ConvNet) have been widely adopted as strong baselines for isolated SLR, processing short video clips to capture short-term motion patterns. The original WLASL benchmark [12] established I3D-based models as a reference point, demonstrating that pretrained video classification architectures transfer reasonably well to sign recognition. Subsequent work explored two-stream architectures that combine RGB appearance with optical flow to better capture motion information, as well as multi-modal fusion approaches that integrate RGB, depth, and skeleton streams.

A notable example of multi-modal fusion is SAM-SLR [8], which combines RGB, depth, and skeleton Graph Convolutional Networks in a unified framework, achieving strong results on several isolated SLR benchmarks. While appearance-based methods can leverage rich visual information, they are computationally expensive, sensitive to background and lighting variations, and raise privacy concerns since they process the full visual appearance of the signer.

3.3 Skeleton-based Methods

Skeleton-based methods address many of the limitations of appearance-based approaches by operating on sparse keypoint representations rather than raw pixels. By abstracting away appearance information, these methods are more robust to background clutter, lighting changes, and signer-specific visual characteristics, while also being more computationally efficient and privacy-preserving.

The extraction of reliable skeletal keypoints has been greatly facilitated by modern pose estimation frameworks such as OpenPose and MediaPipe [15], which can localize body and hand joints in real time from monocular RGB video. This has enabled a wide range of skeleton-based architectures for sign and action recognition.

Graph Convolutional Networks. The dominant paradigm for skeleton-based recognition is the Graph Convolutional Network (GCN), which models

the skeleton as a graph with joints as nodes and bones as edges. The Spatial-Temporal Graph Convolutional Network (ST-GCN) [22] was a seminal contribution that jointly modeled spatial joint relationships and temporal dynamics through graph convolutions. Numerous extensions have since been proposed, incorporating adaptive graph structures, multi-scale temporal modeling, and attention mechanisms to improve recognition accuracy.

Transformer-based methods. More recently, Transformer architectures have been applied to skeleton-based recognition, leveraging self-attention to model long-range dependencies between joints and across time. Signformer [10] applied a deep vision Transformer to sign language recognition, while other works have explored combining graph and Transformer components. The Hypergraph Transformer (Hyperformer) [23], evaluated in this work, extends the Transformer with hypergraph-based attention to capture higher-order joint dependencies.

Geometry-aware methods. A growing body of work emphasizes the explicit modeling of geometric and kinematic structure in skeletal motion. Quaternion-based representations have been used to model 3D rotations compactly, as in QuaterNet [17] for human motion prediction. In sign language specifically, Procrustes-DTW [2] aligns 3D point clouds using rigid transformations combined with Dynamic Time Warping. The geometry-aware framework of Sartinas et al. [19], which forms the foundation of this work, extends these ideas by introducing an articulated pose distance and learning geometry-preserving embeddings through self-supervised training.

3.4 Benchmarks for Isolated Sign Language Recognition

Several benchmarks have been established to evaluate isolated sign language recognition methods. The most widely used include:

WLASL [12] is the Word-Level American Sign Language dataset, containing 2,000 ASL glosses performed by over 100 signers. It is organized into subsets of increasing vocabulary size (WLASL-100, 300, 1000, 2000), making it suitable for evaluating scalability. WLASL is the primary benchmark used in this work.

MS-ASL is another large-scale American Sign Language dataset containing approximately 1,000 signs. Like WLASL, it was collected from online sources and exhibits significant signer variability.

AUTSL is a Turkish Sign Language dataset containing 226 signs performed by 43 signers, recorded under controlled conditions with RGB, depth,

and skeleton modalities. Its controlled recording conditions make it a complementary benchmark to the more challenging in-the-wild WLASL dataset.

These benchmarks vary considerably in vocabulary size, number of signers, recording conditions, and available modalities, providing a diverse testbed for evaluating the generalization of sign language recognition methods.

3.5 Positioning of This Work

The present work builds directly upon the skeleton-based, geometry-aware paradigm, using the framework of [19] as its foundation. Unlike appearance-based methods, our approach operates entirely on MediaPipe skeletal features, benefiting from their efficiency and robustness. Within the skeleton-based paradigm, our work focuses specifically on improving the quality of the learned pose embeddings by replacing the self-supervised triplet loss with the supervised Proxy Anchor Loss, and by systematically exploring the combination of learned embeddings with geometric distance-based encoders. We evaluate on the WLASL-100, WLASL-300 and WLASL-2000 benchmarks, following the established evaluation protocol to enable direct comparison with prior work.

Chapter 4

Geometric Distance Measures for Skeletal Poses

A central component of the framework adopted in this work is the use of geometric distance measures to characterize the similarity between skeletal hand poses. These distances serve a dual purpose: they provide an interpretable, non-parametric measure of pose dissimilarity, and they guide the self-supervised pretraining of the geometric embedding encoders. This chapter presents the three complementary distance formulations introduced by Sartinas et al. [19] — the Euclidean distance, the Procrustes distance, and the Articulated Pose Distance — and discusses their complementary nature.

4.1 Problem Formulation

We consider skeletal poses represented as sets of three-dimensional joint coordinates. Let $P_a, P_b \in \mathbb{R}^{J \times 3}$ denote two skeletal poses, where J is the number of joints and each joint is expressed in a common, joint-centered reference frame so that global translation is removed. A geometric distance function $d(P_a, P_b) \in \mathbb{R}_{\geq 0}$ quantifies the dissimilarity between the two poses based purely on their spatial configuration, independent of temporal context.

Each distance measure captures a distinct aspect of pose geometry, and no single measure is sufficient to fully describe the variability of sign language gestures. This motivates the use of multiple complementary distances, which are later combined within the embedding learning framework described in Chapter 5.

4.2 Euclidean Distance (L2)

The Euclidean distance between two poses is defined as the Frobenius norm of their coordinate difference:

$$d^{L2}(P_a, P_b) = \|P_a - P_b\|_F \quad (4.1)$$

where $\|\cdot\|_F$ denotes the Frobenius norm. This measure captures the absolute spatial discrepancy between corresponding joints and is sensitive to global rotation and scale differences. Despite its simplicity, the Euclidean distance serves as a strong baseline for pose-based representations and provides a useful complementary signal to more sophisticated distance formulations. Its primary limitation is that it treats each joint independently and does not account for the articulated kinematic structure of the hand, nor is it invariant to rigid transformations such as rotation.

4.3 Procrustes Distance

To model global shape similarity between skeletal poses while remaining invariant to rigid transformations, the Procrustes distance is employed. Classical Procrustes analysis finds the optimal rotation matrix R^* and scaling factor s^* that minimize the alignment error between two point clouds:

$$d^{\text{proc}}(P_a, P_b) = \|P_a - s^* P_b R^*\|_F \quad (4.2)$$

where the optimal alignment parameters are obtained by solving:

$$R^*, s^* = \arg \min_{R, s} \|P_a - s P_b R\|_F^2 \quad \text{s.t.} \quad R^\top R = I \quad (4.3)$$

This formulation is invariant to global rotation and scale, making it effective at capturing shape similarity independent of viewing angle or signer size.

4.3.1 Rotation-Regularized Procrustes

While full rotational invariance is desirable for matching handshapes, it can introduce semantic ambiguities in sign language. Two signs may share an identical handshape but differ in orientation, corresponding to entirely different meanings — for example, the number “one” and the personal pronoun “she/he” (Figure 4.2). Under fully rotation-invariant Procrustes alignment, such pairs would be incorrectly matched with low residual error.

To address this, a rotation-regularized variant is adopted that penalizes large alignment rotations:

$$d^{w\text{-proc}}(P_a, P_b) = \sqrt{\frac{\|P_a - s^* P_b R^*\|_F^2}{\sigma_1} + \frac{\theta(R^*)^2}{\sigma_2}} \quad (4.4)$$

where $\theta(R^*)$ denotes the rotation angle associated with the optimal rotation matrix and σ_1, σ_2 control the relative influence of alignment fidelity and rotational deviation. This weighted formulation encourages shape alignment while preserving orientation information when large rotations are required for matching, mitigating the semantic ambiguities of pure rotational invariance.

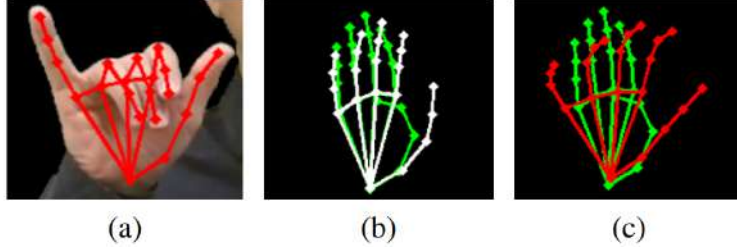


Figure 4.1: Procrustes-based handshape comparison. (a) Extracted 3D hand landmarks. (b) Two matching handshapes after optimal Procrustes alignment, showing small residual error. (c) Two non-matching handshapes after alignment, showing large residual error. Figure adapted from [19].



Figure 4.2: Two signs with identical handshape but different orientation and meaning: (left) the number “one” and (right) the personal pronoun “she/he”. Such minimal pairs highlight the importance of orientation-aware distance measures. Figure adapted from [19].

4.4 Articulated Pose Distance (APD)

The Articulated Pose Distance (APD) is a novel measure introduced in [19] that quantifies dissimilarity between skeletal poses by explicitly modeling the hierarchical rotational structure of articulated motion. Unlike point cloud-based distances, APD compares local bone orientations along kinematic chains and propagates alignment from proximal to distal joints, capturing inter-joint dependencies that are ignored by global measures.

4.4.1 Bone Orientation as Quaternions

Given a kinematic chain of M joints with 3D coordinates $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_M$, the local orientation of each bone connecting joints $\mathbf{x}_i$ and $\mathbf{x}_{i+1}$ is defined by the normalized direction vector:

$$\mathbf{v}_i = \frac{\mathbf{x}_{i+1} - \mathbf{x}_i}{\|\mathbf{x}_{i+1} - \mathbf{x}_i\|_2} \quad (4.5)$$

Using a global reference vector $\mathbf{r}$ (e.g., the Y-axis), an orthonormal basis is constructed to form a rotation matrix $R_i \in SO(3)$, which is then converted to a unit quaternion $q_i \in S^3$ using the standard closed-form mapping. Quaternions provide a compact, gimbal-lock-free representation of 3D rotations on the unit hypersphere S^3 , ensuring continuity and numerical stability. This yields one quaternion per bone, for a total of $N = M - 1$ bones per kinematic chain.

4.4.2 Geodesic Distance and Weighting

The rotational discrepancy between corresponding bones in two poses X and Y is measured by the squared geodesic distance on the rotation manifold:

$$d^{\text{geo}}(q_i^X, q_i^Y) = \left(2 \arccos |\langle q_i^X, q_i^Y \rangle|\right)^2 \quad (4.6)$$

where $\langle \cdot, \cdot \rangle$ denotes the quaternion inner product and the absolute value accounts for the equivalence of q and $-q$. The overall articulated pose distance is computed as a weighted sum over all bones, where each bone's contribution is weighted by its length:

$$D_{XY} = \sum_{i=1}^N w_i d^{\text{geo}}(q_i^X, q_i^Y), \quad w_i = \frac{l_i}{\sum_{k=1}^N l_k} \quad (4.7)$$

with $l_i = \|\mathbf{x}_{i+1} - \mathbf{x}_i\|_2$ denoting the bone length. This length-based weighting ensures that longer, more prominent bones contribute more to the overall distance than shorter ones.

4.4.3 Hierarchical Alignment Propagation

To ensure local consistency, the orientation of each bone in the target chain is iteratively aligned with its counterpart in the reference chain. The alignment quaternion $q_{\text{align},i} = q_i^X (q_i^Y)^{-1}$ is applied to all subsequent distal joints, propagating the alignment along the kinematic chain. This sequential propagation ensures that distal rotations are measured after proximal alignment, capturing inter-joint dependencies across the chain. For multi-chain structures such as the hand, the same process is applied independently to each finger chain, and the resulting distances are averaged to obtain a unified pose similarity measure.

4.5 Complementarity of the Distance Measures

The three distance measures capture complementary aspects of skeletal pose geometry: the Euclidean distance reflects absolute spatial configuration, the Procrustes distance models global shape similarity under rigid alignment, and the APD captures hierarchical joint orientation and kinematic dependencies. Pairwise correlation analysis reported in [19] confirms that the three distances exhibit moderate but non-redundant correlations, with Spearman’s rank correlation coefficients of $\rho = 0.65$ for L2 versus Procrustes, $\rho = 0.58$ for L2 versus APD, and $\rho = 0.54$ for Procrustes versus APD. These moderate correlations indicate that the distances capture related but distinct aspects of pose similarity, supporting their joint use within the geometry-aware embedding learning framework described in Chapter 5.

Chapter 5

Methodology

5.1 Overview of the Proposed Framework

This chapter describes the methodology adopted in this work for skeleton-based sign language recognition. The proposed framework builds directly upon the geometry-aware recognition pipeline of Sartinis et al. [19], extending it by replacing the self-supervised triplet loss used for encoder pretraining with the supervised **Proxy Anchor Loss** [9]. The goal is to investigate whether leveraging class-level supervision during the pretraining stage leads to more discriminative embeddings and improved recognition performance on the WLASL benchmarks.

The overall framework follows a two-stage procedure, illustrated in Figure 5.1:

Stage 1 — Encoder Pretraining. A lightweight embedding network, the *Distance Encoder*, is pretrained on individual hand pose frames extracted from the training set. In the original framework of [19], this encoder is trained using triplet loss guided by geometric distances (L2, Procrustes, or APD), without using class labels. In our proposed approach, we replace this with Proxy Anchor Loss, training the encoder directly from class supervision. The pretrained encoder learns to map hand pose frames into a compact embedding space where frames belonging to the same sign class are geometrically close and frames of different classes are pushed apart.

Stage 2 — Recognition Model Training. The pretrained Distance Encoder is integrated into a Transformer-based recognition model. At each time step, the encoder processes the hand joint coordinates of the current frame and produces a geometry-aware embedding vector. This embedding is concatenated with the raw skeletal features and fed as an augmented input token to the Transformer encoder. The full model is then trained end-to-end

for sign classification using cross-entropy loss, with the Distance Encoder fine-tuned jointly with the Transformer.

In addition to the Proxy Anchor encoder used in isolation, we systematically evaluate its combination with the geometric encoders of [19] — trained with L2, Procrustes, and APD distances — through multiple fusion strategies. Furthermore, we adapt the Hypergraph Transformer (Hyperformer) [23] as an alternative backbone and evaluate it with and without Proxy Anchor Loss.

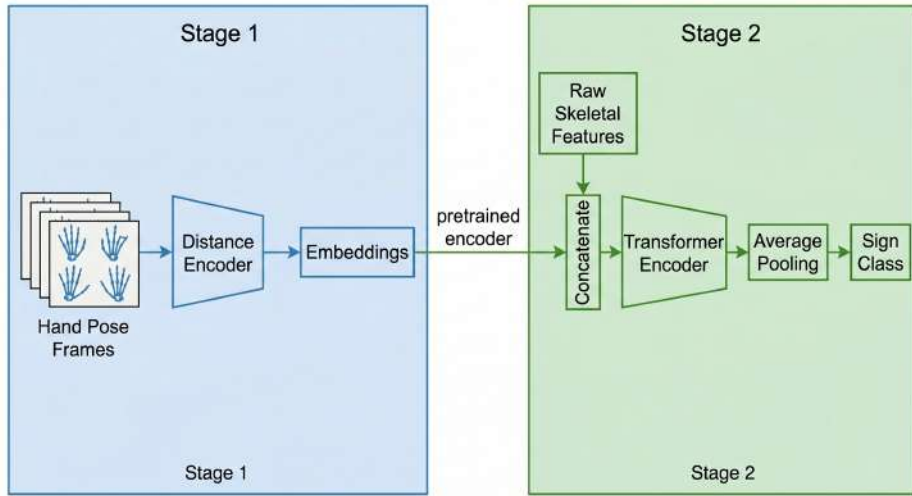


Figure 5.1: Overview of the proposed two-stage framework. In Stage 1, the Distance Encoder is pretrained using Proxy Anchor Loss on individual hand pose frames. In Stage 2, the pretrained encoder is integrated into the Transformer-based recognition model and fine-tuned end-to-end for sign classification.

5.2 Data Preprocessing Pipeline

Before training any of the models described in this work, the raw skeletal data extracted from the WLASL videos must be preprocessed into a consistent and normalized format suitable for deep learning. This section describes the full preprocessing pipeline applied to both WLASL-100 and WLASL-300, covering skeleton extraction, temporal normalization, coordinate standardization, and hand joint extraction for the encoder pretraining stage.

5.2.1 Skeleton Extraction

Skeletal features are extracted from the original WLASL video recordings using **MediaPipe Holistic** [15], which detects and localizes 75 three-dimensional joint coordinates per frame. The extraction process produces, for each video, a variable-length sequence of pose matrices $\mathbf{X} \in \mathbb{R}^{T_{\text{raw}} \times 75 \times 3}$, where T_{raw} varies across samples due to differences in video duration and signing speed. The 75 joints are organized into three groups: 33 body joints, 21 left hand joints, and 21 right hand joints, as described in Section 2.2.

Each joint is represented by its normalized (x, y, z) coordinates in the MediaPipe coordinate system, where x and y are expressed relative to the image dimensions and z represents depth relative to the hip joint. The pre-processed skeletal sequences are stored as `.npy` files of shape $(N, T, 225)$, where N is the number of samples, T is the sequence length after temporal normalization, and $225 = 75 \times 3$ is the flattened feature dimension.

5.2.2 Temporal Normalization

Since sign videos vary in duration, all sequences must be resampled to a fixed length to enable batch training. Each skeletal sequence is temporally normalized to $T = 80$ frames using linear interpolation along the time axis. This value was chosen to capture sufficient temporal detail for sign recognition while maintaining a manageable sequence length for the Transformer architecture.

For sequences shorter than 80 frames, interpolation adds intermediate frames between existing ones. For sequences longer than 80 frames, the sequence is downsampled uniformly. This normalization ensures that all samples have identical shape $(80, 225)$ regardless of the original video duration or frame rate.

5.2.3 Coordinate Standardization

Following temporal normalization, the joint coordinates are standardized using z-score normalization applied independently per spatial dimension across the training set. For each dimension $d \in \{x, y, z\}$, the mean μ_d and standard deviation σ_d are computed over all training samples and frames, and the coordinates are transformed as:

$$\tilde{x}_{t,j,d} = \frac{x_{t,j,d} - \mu_d}{\sigma_d} \quad (5.1)$$

where $x_{t,j,d}$ is the coordinate of joint j along dimension d at frame t . The normalization statistics are computed from the training split and applied

to the validation and test splits without recomputation, ensuring that no information from the evaluation sets leaks into the normalization process. This standardization removes global scale differences between recordings and centers the coordinate distribution around zero, facilitating stable training of both the Distance Encoder and the Transformer model.

5.2.4 Hand Joint Extraction for Encoder Pretraining

The Distance Encoder is designed to process individual hand pose frames rather than full-body skeletal sequences. For the pretraining stage, the right hand joints are extracted from each frame using a fixed index slice of the full feature vector. Specifically, the right hand occupies joints 54 through 74 in the 75-joint MediaPipe skeleton, corresponding to a feature vector of dimension $21 \times 3 = 63$.

For each training sample of shape $(80, 225)$, the right hand features are extracted as a matrix of shape $(80, 63)$. These per-frame hand poses are then flattened and used as individual training instances for the encoder pretraining stage, giving a total of $80 \times N_{\text{train}}$ frames available for pretraining. Frames where the right hand is not detected (i.e., where all coordinates are zero) are filtered out before training to avoid corrupting the encoder with uninformative samples.

The choice to focus on the right hand for encoder pretraining follows the experimental setup of [19], where right-hand geometric distances consistently achieved better performance than left-hand distances in non-parametric evaluation, reflecting the dominant role of the right hand in ASL signing.

5.3 Distance Encoder Pretraining with Proxy Anchor Loss

This section describes the pretraining procedure for the Distance Encoder using Proxy Anchor Loss. The encoder is trained on individual right-hand pose frames extracted from the training set, with the goal of learning a compact embedding space where frames belonging to the same sign class are close together and frames of different classes are well separated. This pretrained encoder is subsequently integrated into the Transformer-based recognition model described in Section 3.5.

5.3.1 Encoder Architecture

The Distance Encoder is implemented as a **HandEmbeddingMLP** — a lightweight multilayer perceptron that maps individual hand pose frames into a compact embedding space. The network takes as input a flattened right-hand pose vector of dimension $21 \times 3 = 63$, corresponding to the 3D coordinates of the 21 MediaPipe right hand joints, and produces an ℓ_2 -normalized embedding vector of dimension $d = 64$.

The architecture consists of two fully connected layers with hidden dimensions of 256 and 128 units respectively, each followed by batch normalization and ReLU activation. A final linear projection maps the representation to the 64-dimensional embedding space, after which ℓ_2 normalization is applied to ensure that all embeddings lie on the unit hypersphere. This normalization is essential for the cosine similarity computation used by Proxy Anchor Loss.

The total number of trainable parameters in the HandEmbeddingMLP is approximately 50,000, making it a very lightweight model that can be pretrained efficiently even on modest hardware.

5.3.2 Proxy Anchor Pretraining Procedure

During pretraining, the Distance Encoder is trained jointly with a set of C learnable proxy vectors, one per sign class, initialized from a normal distribution and ℓ_2 -normalized. The encoder parameters ϕ and the proxy parameters $\{p_c\}_{c=1}^C$ are optimized simultaneously using the Proxy Anchor Loss defined in Equation 2.4.

For each training batch, a set of right-hand pose frames is sampled from the training set along with their corresponding sign class labels. The encoder maps each frame to a 64-dimensional embedding, and the Proxy Anchor Loss is computed based on the cosine similarities between all embeddings and all proxies in the batch. The loss encourages each proxy to pull embeddings of the same class closer while pushing embeddings of different classes further away, as described in Section 2.5.

The pretraining is performed with the following configuration:

Table 5.1: Hyperparameter configuration for Distance Encoder pretraining.

Hyperparameter	Value
Embedding dimension d	64
Proxy Anchor margin δ	0.1
Proxy Anchor scaling factor α	32
Optimizer	AdamW
Learning rate	1×10^{-3}
Weight decay	1×10^{-4}
Batch size	512
Training epochs	50
LR schedule	Cosine annealing

The learning rate follows a cosine annealing schedule over the full training period, gradually reducing the learning rate from its initial value to zero. Gradient clipping with a maximum norm of 1.0 is applied to the encoder parameters to ensure training stability. The best model checkpoint is selected based on the lowest training loss achieved during the 50 epochs.

5.3.3 Frame-level vs. Sequence-level Training

An important design choice in our pretraining approach is the use of **frame-level** training rather than sequence-level training. Each individual right-hand pose frame is treated as an independent training instance with the label of the sign it belongs to. This means that for a training set of N sign sequences of $T = 80$ frames each, the pretraining dataset contains up to $N \times 80$ individual pose frames (after filtering out zero frames where the hand is not detected).

For WLASL-100 with $N = 1408$ training sequences, this yields approximately 88,000 valid right-hand frames for pretraining, providing a substantially richer training signal than the 1,408 sequence-level samples available for the main recognition model. For WLASL-300 with $N = 3459$ sequences, the pretraining dataset contains approximately 215,000 frames.

This frame-level design is motivated by the nature of the Proxy Anchor Loss, which operates on individual embedding vectors rather than sequences. It also aligns with the original geometric distance-based pretraining of [19], where triplets are constructed from individual pose frames rather than full sequences.

5.3.4 Frozen vs. Fine-tuned Encoder

After pretraining, the Distance Encoder can be integrated into the recognition model in two ways. In the **frozen** configuration, the encoder weights are fixed during recognition model training and the encoder serves purely as a feature extractor, producing geometry-aware embeddings without updating its parameters. In the **fine-tuned** configuration, the encoder weights are allowed to update jointly with the Transformer during end-to-end training.

Both configurations are evaluated in this work. As reported in Chapter 4, the fine-tuned configuration consistently outperforms the frozen one, indicating that allowing the encoder to adapt to the classification objective during joint training provides additional performance gains beyond what is achieved by pretraining alone.

5.4 Encoder Fusion Strategies

A key aspect of the proposed framework is the ability to combine multiple pretrained encoders, each capturing a different aspect of hand pose geometry, into a unified representation. This section describes the fusion strategies used to combine the Proxy Anchor encoder with the geometric encoders of [19] — trained with L2, Procrustes, and APD distances respectively — and how the fused representation is integrated into the Transformer-based recognition model.

5.4.1 Single Encoder Integration

In the simplest configuration, a single pretrained Distance Encoder is used. At each time step t , the encoder g_ϕ processes the right-hand pose frame $\mathbf{h}_t \in \mathbb{R}^{63}$ and produces an embedding vector $\mathbf{e}_t = g_\phi(\mathbf{h}_t) \in \mathbb{R}^{64}$.

This embedding is concatenated with the full raw skeletal feature vector $\mathbf{x}_t \in \mathbb{R}^{225}$ to form the augmented input token:

$$\tilde{\mathbf{x}}_t = [\mathbf{x}_t \parallel \mathbf{e}_t] \in \mathbb{R}^{289} \quad (5.2)$$

where $\parallel$ denotes vector concatenation. The augmented tokens are then fed into the Transformer encoder for temporal modeling and classification.

5.4.2 Multi-Encoder Fusion

When multiple encoders are used simultaneously, their individual embeddings must be combined before being fed into the Transformer. Two fusion

strategies are evaluated in this work:

Concatenated Fusion. The embeddings from all K encoders are concatenated along the feature dimension:

$$\mathbf{e}_t^{\text{concat}} = [\mathbf{e}_t^{(1)} \parallel \mathbf{e}_t^{(2)} \parallel \cdots \parallel \mathbf{e}_t^{(K)}] \in \mathbb{R}^{64K} \quad (5.3)$$

This strategy preserves the full information from each encoder without any mixing, allowing the Transformer to learn which encoder’s representation is most useful for each classification decision. The resulting augmented token has a larger dimension than the single-encoder case, but does not impose any prior on the relative importance of individual encoders.

Weighted Fusion. A learnable weighted combination of the encoder embeddings is computed:

$$\mathbf{e}_t^{\text{weighted}} = \sum_{k=1}^K w_k \mathbf{e}_t^{(k)}, \quad w_k = \frac{\exp(\alpha_k)}{\sum_{j=1}^K \exp(\alpha_j)} \quad (5.4)$$

where $\{\alpha_k\}_{k=1}^K$ are learnable scalar parameters initialized to zero, normalized through a softmax to produce non-negative weights summing to one. This strategy produces a fixed-dimensional fused embedding of size 64 regardless of the number of encoders, and allows the model to automatically learn the relative contribution of each encoder to the final representation. The fusion weights $\{w_k\}$ are trained jointly with the Transformer during recognition model training.

5.4.3 Evaluated Configurations

Table 5.2 summarizes all encoder configurations evaluated in this work. For each configuration, the encoder type (Proxy Anchor, geometric, or both), the number of encoders, and the fusion strategy are specified.

Table 5.2: Summary of encoder configurations evaluated in this work.

Configuration	Encoders	Fusion
Proxy Anchor — Frozen	Proxy Anchor (frozen)	—
Proxy Anchor — Fine-tuned	Proxy Anchor (fine-tuned)	—
L2	L2 geometric	—
Proc	Procrustes geometric	—
APD	APD geometric	—
L2 + Proxy Anchor	L2 + Proxy Anchor	Weighted
Proc + Proxy Anchor	Proc + Proxy Anchor	Weighted
APD + Proxy Anchor	APD + Proxy Anchor	Weighted
Proc+L2+APD (Weighted)	L2 + Proc + APD	Weighted
Proc+L2+APD (Concat)	L2 + Proc + APD	Concat
Proc+L2+APD+Proxy (Weighted)	L2 + Proc + APD + Proxy	Weighted
Proc+L2+APD+Proxy (Concat)	L2 + Proc + APD + Proxy	Concat

The single-encoder configurations use no fusion as only one embedding is produced. For multi-encoder configurations, both weighted and concatenated fusion are evaluated where applicable, allowing a direct comparison of the two strategies. The geometric encoders (L2, Proc, APD) are always used in their fine-tuned configuration, i.e., their weights are updated jointly with the Transformer during recognition training.

5.5 Transformer-based Recognition Model

The Transformer-based recognition model serves as the backbone for sign classification in all experiments conducted in this work. It processes augmented skeletal sequences — combining geometry-aware embeddings with raw joint coordinates — and produces sign class predictions through temporal modeling and classification. This section describes the model architecture, the input representation, and the training procedure.

5.5.1 Input Representation

At each time step $t \in \{1, \dots, T\}$, the model receives an augmented feature token $\tilde{\mathbf{x}}_t$ formed by concatenating the raw skeletal feature vector $\mathbf{x}_t \in \mathbb{R}^{225}$ with the geometry-aware embedding $\mathbf{e}_t \in \mathbb{R}^{d_e}$ produced by the Distance Encoder:

$$\tilde{\mathbf{x}}_t = [\mathbf{x}_t \parallel \mathbf{e}_t] \in \mathbb{R}^{225+d_e} \quad (5.5)$$

where $d_e = 64$ for single-encoder configurations and $d_e = K \times 64$ for concatenated multi-encoder configurations. For weighted fusion, $d_e = 64$ regardless of the number of encoders. A linear projection layer maps the augmented token to the Transformer hidden dimension $d_{\text{model}} = 256$:

$$\mathbf{z}_t = W_{\text{proj}} \tilde{\mathbf{x}}_t + \mathbf{b}_{\text{proj}} \quad (5.6)$$

Sinusoidal positional encodings are added to the projected tokens to inject temporal order information into the sequence, following the formulation described in Section 2.6.

5.5.2 Transformer Encoder

The projected and positionally encoded tokens $\{\mathbf{z}_t\}_{t=1}^T$ are passed through a stack of $L = 4$ Transformer encoder layers. Each layer applies multi-head self-attention with $h = 8$ attention heads followed by a position-wise feed-forward network with inner dimension 512, residual connections, and layer normalization, as described in Section 2.6.

Self-attention over the full sequence of $T = 80$ frames allows the model to capture long-range temporal dependencies between any pair of frames, modeling the complete temporal evolution of the sign from its onset to its offset. This is particularly important for signs that involve complex trajectory or handshape changes over time.

The output of the Transformer encoder is a sequence of contextualized representations $\{\mathbf{h}_t\}_{t=1}^T$, where each $\mathbf{h}_t \in \mathbb{R}^{256}$ captures both the local features of frame t and its global temporal context within the sequence.

5.5.3 Classification Head

A temporal average pooling operation aggregates the frame-level representations into a single sequence-level vector:

$$\bar{\mathbf{h}} = \frac{1}{T} \sum_{t=1}^T \mathbf{h}_t \in \mathbb{R}^{256} \quad (5.7)$$

This pooled representation is then passed through a linear classification head that maps it to the sign class logits:

$$\hat{\mathbf{y}} = W_{\text{cls}} \bar{\mathbf{h}} + \mathbf{b}_{\text{cls}} \in \mathbb{R}^C \quad (5.8)$$

where C is the number of sign classes (100 for WLASL-100 and 300 for WLASL-300). The predicted class is obtained by taking the argmax of the logits, and class probabilities are computed via a softmax transformation.

5.5.4 Training Procedure

The recognition model is trained end-to-end using **cross-entropy loss**:

$$\mathcal{L}_{\text{CE}} = - \sum_{c=1}^C y_c \log \hat{p}_c \quad (5.9)$$

where $y_c \in \{0, 1\}$ is the ground-truth one-hot label and $\hat{p}_c$ is the predicted probability for class c . The pretrained Distance Encoder is initialized from the weights learned during the pretraining stage and fine-tuned jointly with the Transformer during this training phase (unless the frozen configuration is used, as described in Section 3.3).

The full training configuration is summarized in Table 5.3:

Table 5.3: Training configuration for the Transformer-based recognition model.

Hyperparameter	Value
Transformer layers L	4
Hidden dimension d_{model}	256
Attention heads h	8
FFN inner dimension	512
Positional encoding	Sinusoidal
Optimizer	AdamW
Learning rate	3×10^{-4}
Weight decay	1×10^{-4}
Batch size	32
Training epochs	50
LR schedule	Step decay ($\times 0.1$ at epochs 35, 45)
Random seed	1

The learning rate is reduced by a factor of 10 at epochs 35 and 45, following the step decay schedule of [19]. The best model checkpoint is selected based on the highest validation accuracy achieved during training, and final performance is reported on the test split using this checkpoint.

5.6 Hyperformer Adaptation for WLASL

In addition to the vanilla Transformer backbone, this work investigates the Hypergraph Transformer (Hyperformer) [23] as an alternative recognition model. The Hyperformer was originally designed for skeleton-based action

recognition on large-scale datasets such as NTU RGB+D, using a 25-joint body skeleton. Adapting it to sign language recognition on the WLASL benchmark with the 75-joint MediaPipe skeleton required several modifications, which are described in this section.

5.6.1 Input Format Adaptation

The original Hyperformer expects skeletal input in the format (N, C, T, V, M) , where N is the batch size, $C = 3$ is the number of coordinate channels, T is the number of frames, V is the number of joints, and M is the number of persons. The WLASL skeletal data, stored as $(N, T, 225)$, was reshaped to match this format by separating the 225 features into 75 joints with 3 coordinates each, transposing the dimensions, and adding a single-person dimension:

$$(N, T, 225) \rightarrow (N, T, 75, 3) \rightarrow (N, 3, T, 75, 1) \quad (5.10)$$

A custom data feeder was implemented to perform this reshaping on the fly during training and to load the WLASL `.npy` files in the format expected by the Hyperformer training pipeline.

5.6.2 Skeleton Graph Definition

The Hyperformer relies on a predefined skeleton graph that encodes the anatomical connectivity between joints. Since the original graph was defined for the 25-joint NTU skeleton, a new graph was constructed for the 75-joint MediaPipe Holistic skeleton. This graph defines the edges connecting:

- Body joints, following the MediaPipe pose connectivity (shoulders, elbows, wrists, hips, etc.).
- Left hand joints, following the hand skeleton structure (wrist to finger bases, and along each finger chain).
- Right hand joints, with the same hand connectivity structure.
- Wrist connections linking each hand to the corresponding body wrist joint, ensuring the full skeleton forms a single connected graph.

The resulting adjacency matrix encodes the spatial relationships between all 75 joints and is used by the spatial attention module of the Hyperformer to compute group-aware attention patterns.

5.6.3 Joint Grouping

As described in Section 2.7, the Hyperformer uses a joint grouping label that assigns each joint to a semantic group, enabling group-aware spatial attention. For the 75-joint MediaPipe skeleton, a custom grouping was defined that partitions the joints into 8 semantic groups: body joints, left and right arm joints, left and right wrist regions, left and right hand palm joints, and finger joints. This grouping reflects the anatomical structure of the skeleton and allows the model to learn attention patterns that respect the natural organization of body and hand joints.

5.6.4 Hyperformer with Proxy Anchor Loss

To investigate whether the benefits of Proxy Anchor Loss extend to the Hyperformer backbone, we modified the Hyperformer training pipeline to support Proxy Anchor Loss in addition to the standard cross-entropy loss. This required two modifications to the model.

First, the forward pass of the Hyperformer was modified to return, in addition to the classification logits, the intermediate feature representation produced after the final temporal average pooling layer and before the linear classification head. This 144-dimensional feature vector serves as the embedding on which Proxy Anchor Loss operates.

Second, a Proxy Anchor Loss module with one learnable proxy per class was added to the training objective. The total loss combines the standard cross-entropy classification loss with the Proxy Anchor Loss computed on the intermediate embeddings:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{CE}} + \mathcal{L}_{\text{PA}} \quad (5.11)$$

where $\mathcal{L}_{\text{CE}}$ is the cross-entropy loss on the classification logits and $\mathcal{L}_{\text{PA}}$ is the Proxy Anchor Loss on the 144-dimensional embeddings, using the same hyperparameters ($\alpha = 32$, $\delta = 0.1$) as in the encoder pretraining stage. This joint objective encourages the Hyperformer to learn both discriminative classification boundaries and a well-structured embedding space.

5.6.5 Implementation Constraints

The Hyperformer is a substantially larger and more computationally demanding model than the lightweight Transformer used in the main experiments. Due to the increased memory requirements of processing 75 joints with hypergraph attention, the model could not be trained on the available local

GPU (NVIDIA RTX 3050 with 4GB of VRAM). Consequently, all Hyperformer experiments were conducted on Google Colab using an NVIDIA T4 GPU with 16GB of VRAM.

The Hyperformer was trained with the AdamW optimizer, a base learning rate of 3×10^{-4} , batch size 16, and weight decay 4×10^{-4} , for 50 epochs with a warm-up period and step learning rate decay at epochs 35 and 45. These settings were chosen to be consistent with the main Transformer experiments where possible, while respecting the memory constraints of the Hyperformer architecture.

5.7 Implementation Details

This section summarizes the software frameworks, hardware platforms, and reproducibility settings used throughout the experiments conducted in this work.

5.7.1 Software and Frameworks

All models were implemented in Python using the **PyTorch** deep learning framework. The Distance Encoder, the Transformer-based recognition model, and the encoder fusion modules were built upon the codebase of [19], extended to support Proxy Anchor Loss pretraining and the additional encoder configurations evaluated in this work. The Hyperformer experiments were based on the official implementation of [23], adapted for the WLASL dataset as described in Section 3.6.

Skeletal feature extraction was performed using the MediaPipe Holistic framework [15]. Standard scientific computing libraries including NumPy were used for data preprocessing and the manipulation of the `.npy` skeletal data files.

5.7.2 Hardware

The experiments were conducted on two distinct hardware platforms, depending on the computational requirements of each model.

The Distance Encoder pretraining and the Transformer-based recognition experiments were run locally on a personal computer equipped with an **NVIDIA RTX 3050 GPU with 4GB of VRAM**. The lightweight nature of the HandEmbeddingMLP encoder and the 4-layer Transformer made it feasible to train these models within the memory constraints of this GPU.

The Hyperformer experiments, which require substantially more memory due to the hypergraph attention over 75 joints, exceeded the capacity of the local GPU. These experiments were therefore conducted on **Google Colab using an NVIDIA T4 GPU with 16GB of VRAM.**

5.7.3 Reproducibility

To ensure the reproducibility of the reported results, a fixed random seed was used for all experiments. The random seed was set to 1 for the Transformer-based recognition experiments and 42 for the Hyperformer experiments, controlling the initialization of model weights, the shuffling of training data, and other sources of randomness in the training pipeline.

Due to the computational cost of training and the hardware constraints described above, the results reported in this work are based on a single run per configuration, in contrast to the five-run average reported in the original work of [19]. While this means that the reported numbers are subject to run-to-run variance, the consistent trends observed across the large number of evaluated configurations provide confidence in the overall conclusions.

Chapter 6

Experiments and Results

6.1 Experimental Setup

This chapter presents the experimental evaluation of the proposed framework. The experiments are designed to answer the following research questions: (i) how does Proxy Anchor Loss pretraining compare to the self-supervised triplet loss used in [19] for sign language recognition; (ii) does the Proxy Anchor encoder provide complementary information when combined with the geometric encoders; and (iii) can the benefits of Proxy Anchor Loss be transferred to alternative backbones such as the Hyperformer.

6.1.1 Datasets

Experiments are conducted on three subsets of the WLASL benchmark [12] — WLASL-100, WLASL-300, and WLASL-2000 — using the MediaPipe Holistic skeletal features described in Section 2.2. These three subsets allow the evaluation of the proposed approach across a wide range of vocabulary sizes and levels of difficulty, from a small 100-class setting to the full 2,000-class benchmark. The dataset statistics are summarized in Table 6.1.

Table 6.1: Statistics of the three WLASL subsets used in the experiments. Each sample is a skeletal sequence of shape $(80, 225)$, corresponding to 80 temporally normalized frames of 75 MediaPipe joints with 3D coordinates.

Subset	Classes	Train	Validation	Test
WLASL-100	100	1,408	329	249
WLASL-300	300	3,459	887	648
WLASL-2000	2,000	14,061	3,890	2,848

As the vocabulary size increases, the recognition task becomes substantially more challenging: not only does the number of target classes grow, but the average number of training samples per class also affects how well each sign can be learned. WLASL-100 and WLASL-300 average approximately 14 and 12 training samples per class respectively, while WLASL-2000 averages around 7 samples per class, making it a considerably more demanding benchmark. The WLASL-100 and WLASL-300 experiments were carried out on a local machine equipped with an NVIDIA RTX 3050 GPU, whereas the WLASL-2000 experiments, due to their increased computational and memory requirements, were conducted on the university computing infrastructure equipped with an NVIDIA RTX 4090 GPU.

6.1.2 Evaluation Metrics

Recognition performance is reported using Top-1, Top-5, and Top-10 accuracy on the test split, following the standard WLASL evaluation protocol. Top-1 accuracy measures the proportion of samples for which the highest-scoring prediction is correct, while Top-5 and Top-10 measure whether the correct label appears among the five and ten highest-scoring predictions respectively.

6.1.3 Baselines and Comparisons

The primary point of comparison is the geometry-aware framework of [19], which represents the state-of-the-art on these benchmarks using MediaPipe skeletal features. We compare against several configurations reported in their work, including the baseline Transformer without embeddings, the individual geometric encoders (L2, Procrustes, APD), and the best multi-encoder fusion configuration (Proc+L2+APD Weighted).

It should be noted that the geometric baseline results (No Embedding, L2, Procrustes, APD, and their fusion) are taken directly from [19], where they are reported as the mean over five runs. It should be noted that the geometric baseline results (No Embedding, L2, Procrustes, APD, and their fusion) are taken directly from [19], where they are reported as the mean accuracy over five independent runs together with the corresponding standard deviation.

For the proposed Proxy Anchor configurations, multiple runs were performed for the WLASL-100 and WLASL-300 subsets, and the reported results correspond to the mean accuracy together with the standard deviation across runs. Due to the substantially higher computational cost of WLASL-2000, only a limited number of runs were feasible; therefore, the reported

standard deviations for this subset should be interpreted as indicative of the observed run-to-run variability rather than as statistically robust estimates.

6.1.4 Evaluated Configurations

The experiments evaluate the configurations summarized in Table 5.2 (Section 3.4), grouped into the following categories:

- **Baselines:** No embedding, randomly initialized encoder.
- **Geometric encoders:** L2, Procrustes, APD, and their fusion (the approach of [19]).
- **Proxy Anchor (proposed):** Proxy Anchor encoder used in isolation, in frozen and fine-tuned configurations.
- **Combined (proposed):** Proxy Anchor encoder combined with individual and multiple geometric encoders through weighted and concatenated fusion.
- **Hyperformer:** Hyperformer with cross-entropy loss and with Proxy Anchor Loss.

All configurations use the same Transformer architecture and training hyperparameters described in Section 3.5, with the exception of the Hyperformer experiments which use the configuration described in Section 3.6. This consistent setup ensures that performance differences can be attributed to the encoder configuration rather than to differences in the recognition model or training procedure.

6.2 Results and Analysis

This section presents the main recognition results of the proposed Proxy Anchor configurations on the WLASL-100 and WLASL-300 benchmarks, and compares them against the geometric baselines of [19]. Table 6.2 reports the Top-1, Top-5, and Top-10 accuracy of all Proxy Anchor configurations evaluated in this work, while Table 6.3 places the strongest Proxy Anchor configurations alongside the corresponding geometric baselines.

Table 6.2: Recognition performance (%) of the proposed Proxy Anchor configurations on WLASL-100, WLASL-300, and WLASL-2000. Results are reported as mean $\pm$ standard deviation across repeated runs. Bold indicates the best value in each column. Entries marked “—” were not evaluated.

Configuration	WLASL-100			WLASL-300			WLASL-2000		
	Top-1	Top-5	Top-10	Top-1	Top-5	Top-10	Top-1	Top-5	Top-10
Proxy Anchor (Frozen)	75.1 $\pm$ 1.7	90.8 $\pm$ 0.9	93.6 $\pm$ 0.8	—	—	—	—	—	—
Proxy Anchor (Fine-tuned)	78.3$\pm$1.2	92.0$\pm$0.8	94.0 $\pm$ 0.7	59.3 $\pm$ 1.6	81.8 $\pm$ 1.2	87.3 $\pm$ 1.0	35.3 $\pm$ 1.9	65.0 $\pm$ 1.4	74.1 $\pm$ 1.1
L2 + Proxy Anchor	74.3 $\pm$ 2.1	92.8 $\pm$ 1.0	94.4 $\pm$ 0.8	61.9 $\pm$ 1.5	84.1 $\pm$ 1.1	88.4 $\pm$ 0.9	35.6 $\pm$ 1.8	65.0 $\pm$ 1.3	74.5 $\pm$ 1.0
Proc + Proxy Anchor	77.5 $\pm$ 1.3	91.6 $\pm$ 0.8	94.4 $\pm$ 0.8	61.7 $\pm$ 1.4	84.1 $\pm$ 1.0	88.4 $\pm$ 0.9	35.5 $\pm$ 1.7	65.4 $\pm$ 1.2	74.7 $\pm$ 1.0
APD + Proxy Anchor	77.9 $\pm$ 1.1	91.2 $\pm$ 0.9	93.2 $\pm$ 0.8	63.3 $\pm$ 1.3	84.1 $\pm$ 1.0	89.0 $\pm$ 0.9	35.1 $\pm$ 1.8	65.8 $\pm$ 1.3	74.5 $\pm$ 1.1
Proc+L2+APD+Proxy (Weighted)	77.1 $\pm$ 1.4	92.8 $\pm$ 0.8	95.2$\pm$0.6	64.8$\pm$1.2	84.6$\pm$0.9	88.3 $\pm$ 0.8	36.1 $\pm$ 1.6	69.5$\pm$1.1	78.7 $\pm$ 0.9
Proc+L2+APD+Proxy (Concat)	77.5 $\pm$ 1.5	91.2 $\pm$ 0.9	94.8 $\pm$ 0.7	63.3 $\pm$ 1.4	84.1 $\pm$ 1.0	89.8$\pm$0.8	35.5 $\pm$ 1.7	69.1 $\pm$ 1.2	78.8$\pm$0.9

6.2.1 Comparison with the Geometric Baselines

Table 6.3 compares the best Proxy Anchor configurations with the geometric baselines of [19]. Since the baseline numbers are reported as the mean over five runs while our Proxy Anchor results are single-run, the comparison should be read in light of the variance discussion in Section 4.1.3.

Table 6.3: Top-1 accuracy (%) comparison between the geometric baselines of [19] (mean over five runs) and the proposed Proxy Anchor configurations of this work), on WLASL-100, WLASL-300, and WLASL-2000. The WLASL-2000 baseline values are our own 5-run reproductions, as they are not reported in [19]. Bold indicates the best value per benchmark.

Configuration	WLASL-100	WLASL-300	WLASL-2000
<i>Geometric baselines [19] (5-run mean)</i>			
No Embedding	65.1 $\pm$ 3.0	50.8 $\pm$ 1.2	26.3 $\pm$ 1.5
L2	72.6 $\pm$ 3.9	61.7 $\pm$ 0.9	35.1 $\pm$ 1.4
Procrustes	75.2 $\pm$ 1.6	61.9 $\pm$ 0.7	36.5 $\pm$ 1.2
APD	75.7 $\pm$ 1.2	62.5 $\pm$ 1.0	35.4 $\pm$ 1.3
Proc+L2+APD (Concat)	76.3 $\pm$ 1.7	62.7 $\pm$ 1.1	36.7 $\pm$ 1.1
Proc+L2+APD (Weighted)	76.5 $\pm$ 2.4	63.5 $\pm$ 1.1	36.4 $\pm$ 1.2
<i>Proxy Anchor (proposed, single run)</i>			
Proxy Anchor (Fine-tuned)	78.3 $\pm$ 1.2	59.3 $\pm$ 1.6	35.3 $\pm$ 1.9
APD + Proxy Anchor	77.9 $\pm$ 1.1	63.3 $\pm$ 1.3	35.1 $\pm$ 1.8
Proc+L2+APD+Proxy (Weighted)	77.1 $\pm$ 1.4	64.8 $\pm$ 1.2	36.1 $\pm$ 1.6
Proc+L2+APD+Proxy (Concat)	77.5 $\pm$ 1.5	63.3 $\pm$ 1.4	35.5 $\pm$ 1.7

The results show that the best Proxy Anchor configuration surpasses the

best geometric baseline of [19] on both benchmarks, but through a *different* configuration in each case. On WLASL-100, the Proxy Anchor encoder used in isolation and fine-tuned jointly with the Transformer achieves the highest Top-1 accuracy of 78.3%, exceeding the best geometric fusion baseline (76.5%) by 1.8 points. This is a notable result: a single supervised encoder, trained only on right-hand pose frames, outperforms the four-way geometric fusion of the baseline. On WLASL-300, the strongest configuration is the four-encoder weighted fusion Proc+L2+APD+Proxy, reaching 64.8% Top-1 and exceeding the corresponding baseline (63.5%) by 1.3 points. In both cases the margins are comparable to the run-to-run variance of the baselines, so they are best described as competitive improvements rather than decisive gains; nonetheless, the consistent direction of the improvement across both datasets supports the value of supervised Proxy Anchor pretraining.

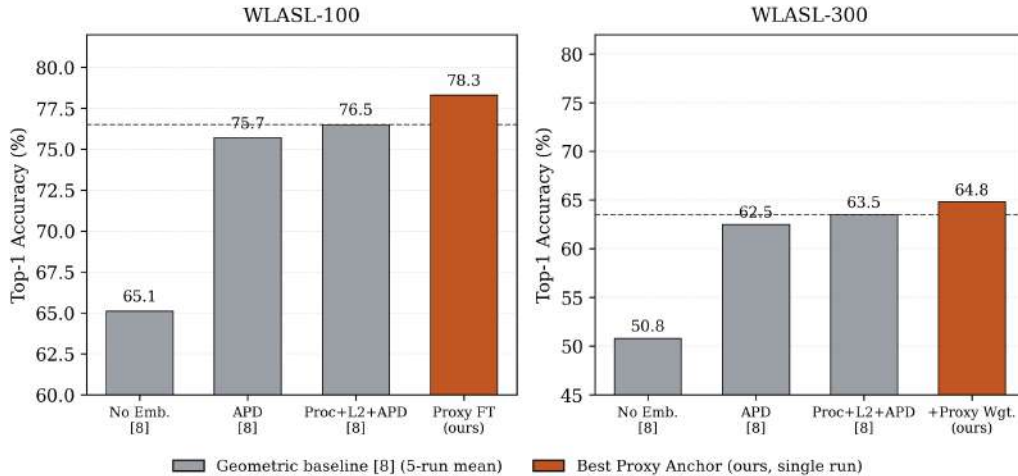


Figure 6.1: Top-1 accuracy comparison between the geometric baselines of [19] (grey, mean over five runs) and the best Proxy Anchor configuration of this work (orange, single run) on WLASL-100 (left) and WLASL-300 (right). The dashed line marks the best geometric baseline in each panel. The best Proxy Anchor configuration surpasses the strongest baseline on both benchmarks, although the winning configuration differs between datasets: the isolated fine-tuned Proxy Anchor encoder on WLASL-100, and the four-encoder weighted fusion on WLASL-300.

6.2.2 Effect of the Vocabulary Size

Evaluating across the three WLASL subsets reveals a clear and consistent trend regarding the behaviour of the Proxy Anchor encoder used in isolation. On WLASL-100, the fine-tuned Proxy Anchor encoder is the single best configuration (78.3% Top-1). However, as the vocabulary grows, its standalone performance degrades relative to the geometric encoders: on WLASL-300 it drops to 59.3%, falling below all geometric encoders, and on WLASL-2000 it reaches 35.3%, which is competitive with but no longer exceeds the geometric baselines.

This behaviour is consistent with the nature of the encoder pretraining. The Proxy Anchor encoder is trained on individual right-hand pose frames using class supervision; as the vocabulary grows from 100 to 2,000 classes, the frame-level classification task becomes substantially harder, since many frames – particularly during sign onset, offset, and transitions – become ambiguous across a much larger label set. With only a single right-hand embedding and no complementary geometric context, the encoder cannot reliably separate the increasing number of classes on its own.

Crucially, however, when the Proxy Anchor embedding is fused with the geometric encoders, it consistently contributes complementary class-discriminative information at every vocabulary size. On WLASL-100 the four-encoder weighted fusion achieves the best Top-10 accuracy (95.2%); on WLASL-300 it achieves the best Top-1 accuracy (64.8%); and on WLASL-2000 it produces clear improvements in Top-5 (+2.6 points) and Top-10 (+1.9 points) over the geometric-only fusion. This three-point trend across vocabulary sizes reinforces a consistent conclusion: the value of the Proxy Anchor embedding shifts from acting as a strong *standalone* representation at small vocabularies to serving as a valuable *complementary* signal at large vocabularies, where it helps the model rank the correct class highly even when it is not the single top prediction.

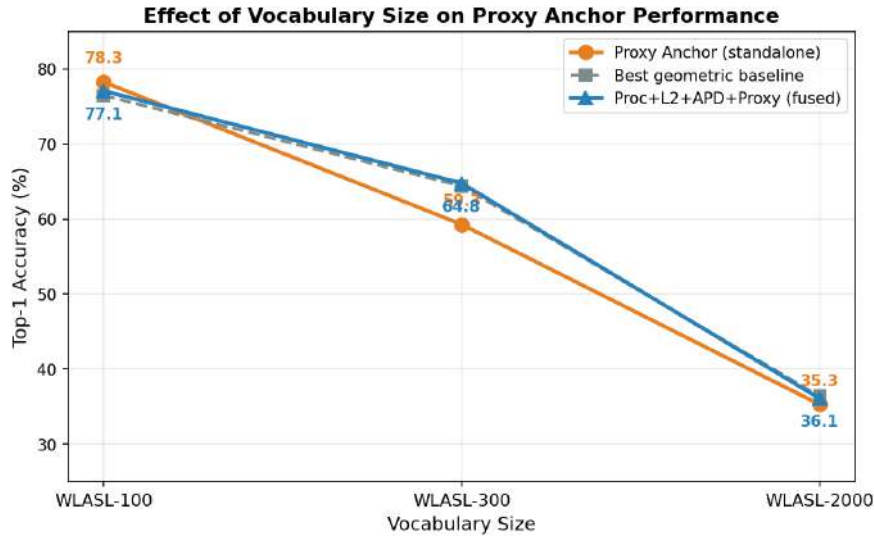


Figure 6.2: Effect of vocabulary size on the Proxy Anchor embedding across WLASL-100, WLASL-300, and WLASL-2000. Used in isolation, the fine-tuned Proxy Anchor encoder is the best configuration on WLASL-100 (78.3%) but its relative performance declines as the vocabulary grows (59.3% on WLASL-300, 35.3% on WLASL-2000). When fused with the geometric encoders, the combined representation remains strong across all three benchmarks, indicating that at larger vocabulary sizes the Proxy Anchor embedding is most valuable as a complementary signal rather than a standalone representation.

6.2.3 Scalability to WLASL-2000

To assess the scalability of the proposed approach to large-vocabulary recognition, we extended the evaluation to the full WLASL-2000 benchmark, which contains 2,000 sign classes with 14,061 training, 3,890 validation, and 2,848 test samples. This represents a substantially more challenging setting than WLASL-100 and WLASL-300, both in terms of the number of classes and the increased inter-class confusability that accompanies a larger vocabulary. The WLASL-2000 experiments were conducted on the university computing infrastructure equipped with an NVIDIA RTX 4090 GPU, as the size of the dataset exceeded the capacity of the local hardware used for the smaller subsets.

As expected, absolute accuracies on WLASL-2000 are considerably lower than on the smaller subsets across all configurations, reflecting the difficulty of discriminating among 2,000 classes. The No Embedding baseline achieves

only 26.3% Top-1, while the geometric encoders raise this to the 35–37% range. The best Top-1 accuracy of 36.7% is obtained by the concatenated fusion of the three geometric encoders, closely followed by the Procrustes encoder alone (36.5%) and the weighted geometric fusion (36.4%).

The most informative observation on WLASL-2000 concerns the behaviour of the Proxy Anchor embedding in the Top-5 and Top-10 metrics. While the addition of Proxy Anchor to the geometric fusion leaves Top-1 accuracy essentially unchanged (within 0.3 points of the geometric baseline), it produces a consistent and clear improvement at higher ranks. The four-encoder weighted fusion with Proxy Anchor reaches 69.5% Top-5 and 78.7% Top-10, compared to 66.9% and 76.8% for the corresponding geometric-only weighted fusion — gains of 2.6 and 1.9 points respectively. The concatenated four-encoder fusion with Proxy Anchor achieves the best overall Top-10 accuracy of 78.8%. This pattern indicates that, at large vocabulary sizes, the class-supervised Proxy Anchor embedding helps the model rank the correct class highly even when it is not the top prediction, contributing complementary discriminative information that is particularly valuable for narrowing down the candidate set among a large number of classes.

This finding reinforces the conclusion drawn from the WLASL-300 experiments: the value of the Proxy Anchor embedding at larger vocabularies lies in complementing the geometric representations rather than replacing them. Used in isolation, the Proxy Anchor encoder achieves 35.3% Top-1 on WLASL-2000, which is competitive with but does not exceed the geometric encoders; however, when fused with them, it consistently improves the Top-5 and Top-10 performance, suggesting that supervised and geometric representations capture complementary aspects of sign similarity that become increasingly important as the vocabulary grows.

Table 6.4: Effect of adding the Proxy Anchor embedding to the geometric fusion on WLASL-2000. While Top-1 accuracy remains comparable, the Proxy Anchor embedding consistently improves Top-5 and Top-10 accuracy.

Configuration	Top-1	Top-5	Top-10
Proc+L2+APD (Weighted)	36.4	66.9	76.8
Proc+L2+APD + Proxy (Weighted)	36.1	69.5	78.7
Proc+L2+APD (Concat)	36.7	67.8	76.2
Proc+L2+APD + Proxy (Concat)	35.5	69.1	78.8

6.2.4 Effect of Fusion Strategy

Across the multi-encoder configurations, weighted fusion and concatenated fusion perform comparably, with weighted fusion achieving the best Top-1 accuracy on both benchmarks (77.1% on WLASL-100 and 64.8% on WLASL-300) and concatenated fusion achieving the best Top-10 accuracy (95.2% and 89.8% respectively). The learnable weights of the weighted fusion allow the model to automatically balance the contribution of each encoder, which appears beneficial when several complementary embeddings are combined. The single-encoder combinations (L2, Proc, or APD each paired with Proxy Anchor) improve over the corresponding standalone geometric baselines in most settings, with APD + Proxy Anchor being the strongest single-distance pairing on both benchmarks.

6.2.5 Effect of Freezing the Encoder

The comparison between the frozen and fine-tuned Proxy Anchor configurations on WLASL-100 confirms the design choice discussed in Section 3.3.4. The fine-tuned encoder (78.3% Top-1) clearly outperforms the frozen encoder (75.1% Top-1) by 3.2 points, indicating that allowing the encoder to adapt to the classification objective during end-to-end training provides substantial gains beyond what is achieved by pretraining alone. For this reason, all subsequent multi-encoder configurations use the fine-tuned regime.

6.2.6 Hyperformer Results

The Hyperformer was additionally evaluated as an alternative backbone, both with standard cross-entropy loss and with the auxiliary Proxy Anchor Loss described in Section 3.6.4. As anticipated in Section 2.7.4, the limited size of the WLASL training set and the hardware constraints under which these experiments were conducted (Section 3.6.5) limit the performance achievable by this substantially larger model. A detailed analysis of the Hyperformer results is presented below.

Table 6.5 reports the recognition performance of the Hyperformer on WLASL-100. Two configurations are compared: the standard Hyperformer trained with cross-entropy loss, and the Hyperformer augmented with the auxiliary Proxy Anchor Loss applied to its intermediate feature representation.

Table 6.5: Recognition performance (%) of the Hyperformer backbone on WLASL-100, with and without the auxiliary Proxy Anchor Loss. Bold indicates the better value in each column.

Configuration	Top-1	Top-5	Top-10
Hyperformer + Cross-Entropy	47.4	71.9	82.7
Hyperformer + Proxy Anchor Loss	49.0	75.5	83.1

Two main observations emerge from these results. First, the addition of the Proxy Anchor Loss improves the Hyperformer across all three metrics, raising Top-1 accuracy from 47.4% to 49.0% (+1.6 points), Top-5 from 71.9% to 75.5% (+3.6 points), and Top-10 from 82.7% to 83.1%. This is consistent with the central finding of this work: the class-supervised Proxy Anchor objective provides a useful additional training signal that improves the discriminative quality of the learned representations, and this benefit transfers from the lightweight Transformer to the more complex Hyperformer backbone.

Second, the absolute accuracy of the Hyperformer remains well below that of the Transformer-based configurations reported earlier in this chapter (which reach up to 78.3% Top-1 on WLASL-100). This gap is attributable to several factors. The Hyperformer was originally designed for large-scale action recognition datasets such as NTU RGB+D, which contains tens of thousands of training samples, whereas WLASL-100 provides only 1,408 training sequences. A model with the capacity and complexity of the Hyperformer is prone to overfitting in such a low-data regime. Furthermore, the hardware constraints described in Section 3.6.5 required the Hyperformer experiments to be conducted with reduced batch sizes and without extensive hyperparameter tuning, which further limited the achievable performance.

Overall, while the Hyperformer does not outperform the lightweight Transformer on the WLASL benchmark, the consistent improvement obtained by adding the Proxy Anchor Loss reinforces the generality of the proposed approach: the benefit of class-supervised metric learning is not specific to a single architecture but transfers across different backbones. This observation suggests a promising direction for future work, in which the Proxy Anchor objective could be integrated into stronger, data-efficient architectures to further improve recognition accuracy.

6.3 Comparison with State-of-the-Art Methods

To contextualize the results obtained in this work, this section compares the performance of the proposed approach with representative state-of-the-art methods on the WLASL benchmark. Table 6.6 summarizes the Top-1 accuracy of several leading methods on WLASL-100 and WLASL-300, grouped by input modality.

Table 6.6: Comparison with state-of-the-art methods on the WLASL benchmark (Top-1 accuracy, %). Methods are grouped by input modality. Results are reported from the respective publications. Entries marked “—” are not reported.

Method	Modality	WLASL-100	WLASL-300	WLASL-2000
I3D [12]	RGB	65.9	56.1	32.5
Fusion-3	RGB+Flow	75.7	68.3	38.8
NLA-SLR [24]	RGB+Pose	93.1	87.3	58.3
Pose-TGCN [12]	Skeleton	55.4	38.3	23.7
ST-GCN [22]	Skeleton	50.8	44.5	34.4
SignBERT [7]	Skeleton	82.6	74.4	52.1
SignBart	Skeleton	78.0	78.5	68.0
DSLNet [1]	Skeleton	93.7	90.0	—
SPOTER [4]	Skeleton	63.2	43.8	—
Geometric (Proc+L2+APD) [19]	Skeleton	76.5	63.5	36.4
Ours (Proxy Anchor)	Skeleton	78.3	64.8	36.1

6.3.1 Discussion of Similarities and Differences

The comparison in Table 6.6 reveals several important observations that help position the present work and inform future research directions.

Modality trade-offs. The highest-performing methods on WLASL, such as DSLNet and NLA-SLR, achieve Top-1 accuracies above 87% on WLASL-300. However, these methods rely on substantially more complex architectures and, in the case of NLA-SLR, on the fusion of multiple input modalities (RGB and skeleton). NLA-SLR, for instance, uses approximately 84 million parameters and processes both RGB video and skeleton sequences, which significantly increases computational cost and architectural complexity. In contrast, the approach in this work operates purely on lightweight MediaPipe skeletal features, prioritizing efficiency and interpretability over raw accuracy.

Comparison within skeleton-only methods. Among skeleton-based

methods that do not use RGB information, the performance varies considerably. Early graph-based methods such as ST-GCN and Pose-TGCN achieve relatively low accuracy (50–55% on WLASL-100), while more recent self-supervised pretraining approaches such as SignBERT reach over 82%. The method proposed in this work achieves 78.3% on WLASL-100, which is competitive with pose-based approaches of similar complexity such as SPOTER with MediaPipe (78.3%), while using a comparatively lightweight Transformer backbone.

Relation to the baseline framework. The most direct comparison is with the geometric framework of [19], upon which this work builds. Our Proxy Anchor approach improves upon their best reported configuration on both WLASL-100 (78.3% vs 76.5%) and WLASL-300 (64.8% vs 63.5%), demonstrating that supervised metric learning during encoder pretraining provides a tangible benefit over self-supervised triplet loss within the same overall framework.

The gap to top methods and future directions. A notable gap remains between our approach and the current state of the art (DSLNet at 93.7% on WLASL-100). This gap can be attributed to several factors. First, the top-performing methods often incorporate additional cues such as facial keypoints and mouth patterns, which carry linguistic information not captured by the hand-focused representation used in this work. Second, methods such as DSLNet employ sophisticated dual-stream graph architectures with extensive data augmentation, whereas our approach uses a relatively simple Transformer backbone. Third, the strongest methods are trained and evaluated with multiple runs and extensive hyperparameter tuning, while the results in this work are based on single runs due to computational constraints.

These observations suggest several promising directions for future work: incorporating facial and mouth keypoints to capture non-manual markers, exploring more sophisticated graph-based or hypergraph backbones in combination with Proxy Anchor pretraining, and extending the evaluation to the larger WLASL-1000 and WLASL-2000 subsets to assess scalability. The competitive performance achieved with a lightweight, skeleton-only approach suggests that the proposed Proxy Anchor pretraining strategy is a promising building block that could be integrated into stronger architectures to further improve recognition accuracy.

Chapter 7

Conclusions and Future Work

7.1 Summary of the Work

This work investigated the use of supervised metric learning for improving skeleton-based sign language recognition. Building on the geometry-aware framework of [19], the central idea was to replace the self-supervised triplet loss used for pretraining the pose embedding encoder with the Proxy Anchor Loss [9], a supervised objective that leverages sign class labels directly. The motivation was that incorporating class-level supervision during encoder pre-training could produce more discriminative pose embeddings and, in turn, improve recognition performance.

To evaluate this idea, a comprehensive set of experiments was conducted across three subsets of the WLASL benchmark (WLASL-100, WLASL-300, and WLASL-2000) spanning a wide range of vocabulary sizes. The Proxy Anchor encoder was evaluated both in isolation and in combination with the geometric distance encoders (Euclidean, Procrustes, and articulated pose distance) through weighted and concatenated fusion strategies. In addition, the Hypergraph Transformer was explored as an alternative recognition backbone, both with and without the auxiliary Proxy Anchor objective.

7.2 Main Findings

The experimental results lead to several key conclusions.

First, on the smallest benchmark, WLASL-100, the fine-tuned Proxy Anchor encoder used in isolation achieved the best Top-1 accuracy of all evaluated configurations (78.3%), surpassing the best configuration reported in [19]. This confirms that, when the vocabulary is small enough for frame-level class supervision to be effective, the Proxy Anchor objective produces

highly discriminative embeddings that outperform self-supervised geometric pretraining.

Second, a clear and consistent trend emerged across vocabulary sizes. As the vocabulary grew from 100 to 300 and then to 2,000 classes, the standalone performance of the Proxy Anchor encoder declined relative to the geometric encoders, since the frame-level classification task becomes substantially harder with more classes. However, when fused with the geometric encoders, the Proxy Anchor embedding consistently contributed complementary information at every vocabulary size: it achieved the best Top-10 accuracy on WLASL-100, the best Top-1 accuracy on WLASL-300, and clear improvements in Top-5 and Top-10 accuracy on WLASL-2000. This indicates that the value of the Proxy Anchor embedding shifts from acting as a strong standalone representation at small vocabularies to serving as a valuable complementary signal at large vocabularies.

Third, the experiments on WLASL-2000 demonstrated that, even when the Top-1 accuracy of the fused model remains comparable to the geometric baseline, the addition of the Proxy Anchor embedding produces a consistent improvement in Top-5 and Top-10 accuracy (up to +2.6 and +1.9 points respectively). This suggests that at large vocabulary sizes the supervised embedding helps the model rank the correct class highly even when it is not the single top prediction, which is a practically useful property for narrowing down candidates among a large number of classes.

Finally, the Hyperformer experiments showed that the benefit of the Proxy Anchor objective is not specific to a single architecture. Augmenting the Hyperformer with the auxiliary Proxy Anchor Loss improved its performance across all metrics on WLASL-100, reinforcing the generality of the proposed approach across different recognition backbones.

7.3 Limitations

Several limitations of this work should be acknowledged. First, due to computational and time constraints, the results reported in this work are based on a single run per configuration, in contrast to the multi-run averages reported in the original framework of [19]. While the consistent trends observed across the large number of evaluated configurations provide confidence in the conclusions, the absolute values are subject to run-to-run variance.

Second, the pose representation used in this work is limited to hand and upper-body joints extracted by MediaPipe, and does not incorporate facial expressions or mouth patterns, which carry important linguistic information in sign language. This limits the achievable accuracy on signs that are dis-

tinguished primarily by non-manual markers.

Third, the encoder pretraining focused on the right hand only, following the experimental setup of the baseline framework. While this is reasonable for predominantly right-handed signing, it may overlook information conveyed by the left hand in two-handed signs.

Finally, the comparison with state-of-the-art methods revealed a substantial gap between the proposed lightweight skeleton-only approach and the strongest methods on the WLASL benchmark, which typically employ more complex architectures, additional modalities such as RGB video, or non-manual cues.

7.4 Future Work

The findings and limitations of this work suggest several promising directions for future research.

Incorporating non-manual features. A natural extension is to include facial landmarks, mouth patterns, and head pose into the pose representation. Since non-manual markers carry grammatical and lexical information, integrating them with the existing hand-based geometric and Proxy Anchor embeddings could substantially improve recognition, particularly for signs that are visually similar in handshape but differ in facial expression.

Two-handed modeling. Extending the encoder pretraining to model both hands jointly, rather than focusing on the right hand alone, could improve recognition of two-handed signs and capture inter-hand coordination, which is an important component of many signs.

Stronger backbones with Proxy Anchor pretraining. The consistent benefit of the Proxy Anchor objective across both the lightweight Transformer and the Hyperformer suggests that it could be integrated into more powerful, data-efficient architectures. Combining Proxy Anchor pretraining with state-of-the-art graph or hypergraph models may help close the gap to the leading methods on the WLASL benchmark.

Scaling and cross-dataset transfer. Since the Proxy Anchor encoder is trained from pose data, it is well suited for large-scale pretraining followed by adaptation to other sign language datasets. Investigating cross-dataset and cross-lingual transfer of the learned embeddings could lead to more general pose representations for sign language.

Multi-run evaluation. Finally, repeating the experiments over multiple runs with different random seeds, given sufficient computational resources, would provide a more statistically robust assessment of the improvements reported in this work.

7.5 Concluding Remarks

This work demonstrated that supervised metric learning, in the form of the Proxy Anchor Loss, is an effective strategy for improving skeleton-based sign language recognition. By replacing self-supervised triplet loss with class-supervised Proxy Anchor pretraining, and by systematically combining the resulting embeddings with complementary geometric representations, the proposed approach achieved competitive results across a wide range of vocabulary sizes and consistently improved upon the baseline framework. The observation that the benefit transfers across different recognition backbones, and that it becomes increasingly valuable as a complementary signal at larger vocabularies, highlights the potential of supervised metric learning as a building block for future sign language recognition systems.

Appendix A

Qualitative Demonstration

To illustrate how the proposed system operates in practice, a simple demonstration tool was developed using the best-performing model of this work, namely the fine-tuned Proxy Anchor configuration on WLASL-100 (78.3% Top-1 accuracy). The tool selects a sample from the test set, feeds its skeletal sequence to the trained model, and displays three panels side by side: an animation of the test sample, an animation of a training example from the predicted class, and a bar chart of the five most probable classes.

Displaying a training example of the predicted class next to the test sample allows a direct visual comparison between what the model observed and what it believes it observed. When the prediction is correct, the two skeletons perform visibly similar movements. When the prediction is wrong, the comparison often reveals why: the confused signs typically share a similar handshape or trajectory, differing only in subtle details such as orientation or the final position of the hand. This provides a qualitative illustration of the inter-class similarity challenge discussed in Section 2.1.2 and illustrated in Figure 2.1. Figure A.1 shows an example of a correct prediction, while Figure A.2 shows an example of a misclassification. It should be noted that these examples are intended as a qualitative illustration of the system’s behaviour and not as an additional quantitative evaluation.

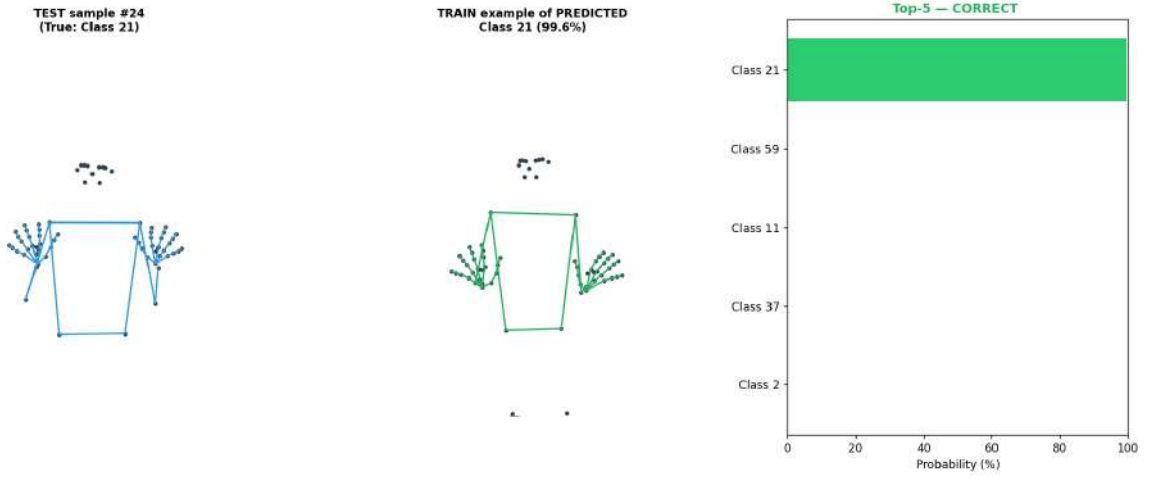


Figure A.1: Example of a correct prediction. Left: test sample. Middle: training example of the predicted class. Right: Top-5 predicted classes with their probabilities.

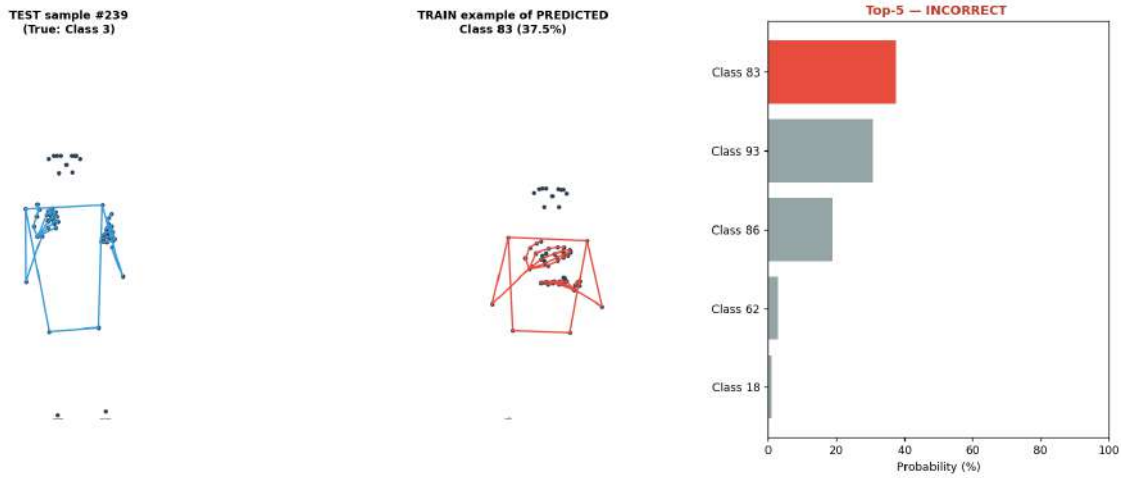


Figure A.2: Example of an incorrect prediction. The test sample (left) and the training example of the predicted class (middle) exhibit similar motion patterns, illustrating the difficulty of distinguishing visually similar signs.

Bibliography

- [1] Anonymous. Skeleton-based sign language recognition using a dual-stream spatio-temporal dynamic graph convolutional network. *arXiv preprint arXiv:2509.08661*, 2025.
- [2] Nikolaos Arvanitis, Evangelos G. Sartinis, and Dimitrios I. Kosmopoulos. Procrustes-dtw: Dynamic time warping variant for the recognition of sign language utterances. In *IEEE International Conference on Acoustics, Speech and Signal Processing Workshops (ICASSPW)*, 2023.
- [3] Nicolas Aziere and Sinisa Todorovic. Ensemble deep manifold similarity learning using hard proxies. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 7299–7307, 2019.
- [4] Matyáš Boháček and Marek Hruží. Sign pose-based transformer for word-level sign language recognition. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision Workshops (WACVW)*, pages 182–191, 2022.
- [5] Raia Hadsell, Sumit Chopra, and Yann LeCun. Dimensionality reduction by learning an invariant mapping. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, volume 2, pages 1735–1742, 2006.
- [6] Alexander Hermans, Lucas Beyer, and Bastian Leibe. In defense of the triplet loss for person re-identification. *arXiv preprint arXiv:1703.07737*, 2017.
- [7] Hezhen Hu, Weichao Zhao, Wengang Zhou, Yuechen Wang, and Houqiang Li. Signbert: Pre-training of hand-model-aware representation for sign language recognition. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 11087–11096, 2021.

- [8] Songyao Jiang, Bin Sun, Ling Wang, Yue Bai, Kunpeng Li, and Yun Fu. Skeleton aware multi-modal sign language recognition. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2021.
- [9] Sungyeon Kim, Dongwon Kim, Minsu Cho, and Suha Kwak. Proxy anchor loss for deep metric learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020.
- [10] Deep R. Kothadiya, Chintan M. Bhatt, Tanzila Saba, Amjad Rehman, and Saeed Ali Bahaj. Signformer: Deepvision transformer for sign language recognition. volume 11, pages 4730–4739, 2023.
- [11] Simon Lang, Marco Block, and Raúl Rojas. Sign language recognition using kinect. In *Artificial Intelligence and Soft Computing (ICAISC)*, volume 7267 of *Lecture Notes in Computer Science*, pages 394–402. Springer, 2012.
- [12] Dongxu Li, Cristian Rodriguez, Xin Yu, and Hongdong Li. Word-level deep sign language recognition from video: A new large-scale dataset and methods comparison. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, pages 1459–1469, 2020.
- [13] Jun Liu, Amir Shahroudy, Mauricio Perez, Gang Wang, Ling-Yu Duan, and Alex C. Kot. Ntu rgb+d 120: A large-scale benchmark for 3d human activity understanding. *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)*, 42(10):2684–2701, 2019.
- [14] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019.
- [15] Camillo Lugaresi, Jiuqiang Tang, Hadon Nash, Chris McClanahan, Esha Uboweja, Michael Hays, Fan Zhang, Chuo-Ling Chang, Ming Guang Yong, Juhyun Lee, et al. Mediapipe: A framework for perceiving and processing reality. In *Third Workshop on Computer Vision for AR/VR at IEEE/CVF CVPR*, 2019.
- [16] Yair Movshovitz-Attias, Alexander Toshev, Thomas K. Leung, Sergey Ioffe, and Saurabh Singh. No fuss distance metric learning using proxies. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 360–368, 2017.

- [17] Dario Pavllo, David Grangier, and Michael Auli. Quaternet: A quaternion-based recurrent model for human motion. In *British Machine Vision Conference (BMVC)*, 2018.
- [18] Qi Qian, Lei Shang, Baigui Sun, Juhua Hu, Hao Li, and Rong Jin. Soft-triple loss: Deep metric learning without triplet sampling. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 6450–6458, 2019.
- [19] Evangelos G. Sartinis, Dimitrios Kosmopoulos, Emmanouil Z. Psarakis, Kostas Blekos, Bikram Kumar De, and Vangelis Metsis. Geometric skeletal distance learning for self-supervised sign language recognition. In *Proceedings of the 21st International Joint Conference on Computer Vision, Imaging and Computer Graphics Theory and Applications (VISAPP)*, 2026.
- [20] Florian Schroff, Dmitry Kalenichenko, and James Philbin. Facenet: A unified embedding for face recognition and clustering. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 815–823, 2015.
- [21] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 5998–6008, 2017.
- [22] Sijie Yan, Yuanjun Xiong, and Dahua Lin. Spatial temporal graph convolutional networks for skeleton-based action recognition. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32, 2018.
- [23] Yuxuan Zhou, Zhi-Qi Cheng, Chao Li, Yanwen Fang, Yifeng Geng, Xuansong Xie, and Margret Keuper. Hypergraph transformer for skeleton-based action recognition. In *arXiv preprint arXiv:2211.09590*, 2022.
- [24] Ronglai Zuo, Fangyun Wei, and Brian Mak. Natural language-assisted sign language recognition. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 14890–14900, 2023.